

Documentacao Tecnica da API

Comparador de Precos - Backend Node.js, Express, Prisma e MySQL

Objetivo do documento

Este PDF explica a API codigo por codigo: para que serve cada arquivo, como as partes se conectam, como os dados entram, como sao validados, como chegam ao banco e como o app Flutter pode consumir as rotas.

Camada	Responsabilidade
Routes	Definem URL, metodo HTTP e validacoes antes do controller.
Controllers	Recebem Request/Response e chamam services.
Services	Concentram regras de negocio e acesso ao Prisma.
DTOs	Validam dados de entrada.
Providers	Conectam a API a fontes externas.
Prisma	Modela banco e fornece client tipado.
Errors/Middlewares	Padronizam validacao e resposta de erro.

Fluxo principal

Cliente chama rota HTTP; rota valida dados com DTO; controller chama service; service executa regra; Prisma salva ou consulta banco; resposta volta em JSON. No fluxo externo, provider busca dados fora, service normaliza e importa para products.

Resumo das Rotas

Metodo e rota	Uso
POST /auth/register	Cria usuario, hash e JWT.
POST /auth/login	Autentica usuario.
GET /products	Lista produtos.
GET /products/search	Busca agregada para o app.
GET /products/trending	Produtos em destaque.
GET /products/:id	Detalhe com historico.
GET /products/:id/offers	Ofertas equivalentes por loja.
GET /products/:id/price-history	Historico de precos.
POST /products/:id/price-snapshots	Registra nova captura.
POST /products	Cria produto.
PUT /products/:id	Atualiza produto.
DELETE /products/:id	Remove produto.
GET /external-products/search	Busca fora sem salvar.
POST /external-products/import	Busca fora e importa.

Fluxos de negocio para apresentacao

Cadastro e login

Register valida usuario, gera hash com bcrypt, salva User e devolve JWT. Login busca por email ou nome, compara senha e gera token.

Produto como oferta

Cada Product representa uma oferta em uma loja. Rotas agregadas juntam ofertas equivalentes por name + category.

Historico de precos

Quando currentPrice muda, o preco antigo vai para price_history e previousPrice. Isso alimenta graficos e variacao.

Integracao externa

O app nao fala direto com marketplace. Nossa API consulta provider externo, normaliza e salva no banco.

externalId

Campo unico que evita duplicar produtos importados e permite atualizar preco do mesmo item.

Validacao

DTOs protegem entrada antes do service. Erros conhecidos usam `AppError` e `errorHandler`.

Observacoes tecnicas importantes

- Arquivos de alerta existem mas estao vazios; o banco ja tem Alert para implementacao futura.
- Login usa interface simples; pode evoluir para classe DTO validada.
- externalId esta mapeado fisicamente como externalId no banco; funciona, mas o nome fisico parece typo.
- Falhas da fonte externa retornam 502 para indicar problema no provider.

package.json

Identidade do projeto, scripts npm e dependencias usadas pela API.

Linha	Codigo	Explicacao
1	{	Abre, fecha ou organiza estrutura JSON.
2	"name": "mobile-backend",	Define uma chave de configuracao JSON do projeto.
3	"version": "1.0.0",	Define uma chave de configuracao JSON do projeto.
4	"description": "",	Define uma chave de configuracao JSON do projeto.
5	"main": "index.js",	Define uma chave de configuracao JSON do projeto.
6	"scripts": {	Abre a lista de comandos npm.
7	"build": "tsc",	Compila TypeScript para dist/.
8	"start": "node dist/server.js",	Roda a API compilada em producao.
9	"dev": "ts-node-dev --respawn --transpile-only src/server.ts",	Roda em desenvolvimento com reload automatico.
10	"test": "echo \"Error: no test specified\" && exit 1"	Define uma chave de configuracao JSON do projeto.
11	},	Abre, fecha ou organiza estrutura JSON.
12	"repository": {	Define uma chave de configuracao JSON do projeto.
13	"type": "git",	Define uma chave de configuracao JSON do projeto.
14	"url": "git+https://github.com/GenissonEmilio/mobile-backend.git"	Define uma chave de configuracao JSON do projeto.
15	},	Abre, fecha ou organiza estrutura JSON.
16	"keywords": [],	Define uma chave de configuracao JSON do projeto.
17	"author": "",	Define uma chave de configuracao JSON do projeto.
18	"license": "ISC",	Define uma chave de configuracao JSON do projeto.
19	"type": "commonjs",	Define uma chave de configuracao JSON do projeto.
20	"bugs": {	Define uma chave de configuracao JSON do projeto.
21	"url": "https://github.com/GenissonEmilio/mobile-backend/issues"	Define uma chave de configuracao JSON do projeto.
22	},	Abre, fecha ou organiza estrutura JSON.
23	"homepage": "https://github.com/GenissonEmilio/mobile-backend#readme",	Define uma chave de configuracao JSON do projeto.
24	"dependencies": {	Dependencias necessarias em runtime.
25	"@prisma/client": "^6.4.0",	Define uma chave de configuracao JSON do projeto.
26	"bcryptjs": "^3.0.3",	Define uma chave de configuracao JSON do projeto.
27	"class-transformer": "^0.5.1",	Define uma chave de configuracao JSON do projeto.
28	"class-validator": "^0.15.1",	Define uma chave de configuracao JSON do projeto.
29	"cors": "^2.8.6",	Define uma chave de configuracao JSON do projeto.
30	"dotenv": "^17.4.2",	Define uma chave de configuracao JSON do projeto.

Linha	Codigo	Explicacao
31	"express": "^5.2.1",	Define uma chave de configuracao JSON do projeto.
32	"jsonwebtoken": "^9.0.3",	Define uma chave de configuracao JSON do projeto.
33	"prisma": "^6.4.0",	Define uma chave de configuracao JSON do projeto.
34	"reflect-metadata": "^0.2.2"	Define uma chave de configuracao JSON do projeto.
35	},	Abre, fecha ou organiza estrutura JSON.
36	"devDependencies": {	Dependencias usadas para desenvolvimento/build.
37	"@types/cors": "^2.8.19",	Define uma chave de configuracao JSON do projeto.
38	"@types/express": "^5.0.6",	Define uma chave de configuracao JSON do projeto.
39	"@types/jsonwebtoken": "^9.0.10",	Define uma chave de configuracao JSON do projeto.
40	"@types/node": "^25.9.3",	Define uma chave de configuracao JSON do projeto.
41	"nodemon": "^3.1.14",	Define uma chave de configuracao JSON do projeto.
42	"ts-node-dev": "^2.0.0",	Define uma chave de configuracao JSON do projeto.
43	"typescript": "^6.0.3"	Define uma chave de configuracao JSON do projeto.
44	}	Abre, fecha ou organiza estrutura JSON.
45	}	Abre, fecha ou organiza estrutura JSON.

tsconfig.json

Configuracao do compilador TypeScript.

Linha	Codigo	Explicacao
1	{	Abre, fecha ou organiza estrutura JSON.
2	"compilerOptions": {	Define uma chave de configuracao JSON do projeto.
3	"target": "ES2020",	Define uma chave de configuracao JSON do projeto.
4	"module": "CommonJS",	Define uma chave de configuracao JSON do projeto.
5		Linha em branco para separar blocos e melhorar legibilidade.
6	"rootDir": "./src",	Define uma chave de configuracao JSON do projeto.
7	"outDir": "./dist",	Define uma chave de configuracao JSON do projeto.
8		Linha em branco para separar blocos e melhorar legibilidade.
9	"strict": true,	Define uma chave de configuracao JSON do projeto.
10		Linha em branco para separar blocos e melhorar legibilidade.
11	"esModuleInterop": true,	Define uma chave de configuracao JSON do projeto.
12	"moduleResolution": "node",	Define uma chave de configuracao JSON do projeto.
13	"ignoreDeprecations": "6.0",	Define uma chave de configuracao JSON do projeto.
14		Linha em branco para separar blocos e melhorar legibilidade.
15	"skipLibCheck": true,	Define uma chave de configuracao JSON do projeto.
16	"forceConsistentCasingInFileNames": true,	Define uma chave de configuracao JSON do projeto.
17		Linha em branco para separar blocos e melhorar legibilidade.
18	"sourceMap": true,	Define uma chave de configuracao JSON do projeto.
19		Linha em branco para separar blocos e melhorar legibilidade.
20	"resolveJsonModule": true,	Define uma chave de configuracao JSON do projeto.
21		Linha em branco para separar blocos e melhorar legibilidade.
22	"experimentalDecorators": true,	Define uma chave de configuracao JSON do projeto.
23		Linha em branco para separar blocos e melhorar legibilidade.
24	"emitDecoratorMetadata": true,	Define uma chave de configuracao JSON do projeto.
25		Linha em branco para separar blocos e melhorar legibilidade.
26	"types": ["node"]	Define uma chave de configuracao JSON do projeto.
27	},	Abre, fecha ou organiza estrutura JSON.
28		Linha em branco para separar blocos e melhorar legibilidade.
29	"include": ["src/**/*"],	Define uma chave de configuracao JSON do projeto.
30	"exclude": ["node_modules", "dist"]	Define uma chave de configuracao JSON do projeto.

Linha	Codigo	Explicacao
31	}	Abre, fecha ou organiza estrutura JSON.

.env.example

Modelo das variaveis de ambiente necessarias.

Linha	Codigo	Explicacao
1	PORT=	Variavel de ambiente que deve ser configurada em cada ambiente.
2	DATABASE_URL="sua_url_do_banco"	Variavel de ambiente que deve ser configurada em cada ambiente.
3	JWT_SECRET="sua_chave_secreta"	Variavel de ambiente que deve ser configurada em cada ambiente.
4	EXTERNAL_PRODUCTS_PROVIDER="mercadolivre"	Variavel de ambiente que deve ser configurada em cada ambiente.
5	MERCADO_LIVRE_API_BASE_URL="https://api.mercadolivre.com"	Variavel de ambiente que deve ser configurada em cada ambiente.
6	MERCADO_LIVRE_SITE_ID="MLB"	Variavel de ambiente que deve ser configurada em cada ambiente.

prisma/schema.prisma

Modelo conceitual do banco usado pelo Prisma.

Linha	Codigo	Explicacao
1	// prisma/schema.prisma	Declara campo, tipo ou configuracao do schema.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	generator client {	Configura geracao do Prisma Client.
4	provider = "prisma-client-js"	Declara campo, tipo ou configuracao do schema.
5	}	Declara campo, tipo ou configuracao do schema.
6		Linha em branco para separar blocos e melhorar legibilidade.
7	datasource db {	Configura fonte de dados do banco.
8	provider = "mysql"	Declara campo, tipo ou configuracao do schema.
9	url = env("DATABASE_URL")	Declara campo, tipo ou configuracao do schema.
10	}	Declara campo, tipo ou configuracao do schema.
11		Linha em branco para separar blocos e melhorar legibilidade.
12	model User {	Declara entidade do banco que o Prisma usa como modelo.
13	id Int @id @default(autoincrement())	Define chave primaria.
14	name String	Declara campo, tipo ou configuracao do schema.
15	email String @unique	Impede duplicidade para este campo.
16	passwordHash String @map("password_hash")	Mapeia nome do Prisma para coluna real no banco.
17	createdAt DateTime @default(now()) @map("createdAt")	Mapeia nome do Prisma para coluna real no banco.
18	alerts Alert[]	Declara campo, tipo ou configuracao do schema.
19		Linha em branco para separar blocos e melhorar legibilidade.
20	@@map("users")	Mapeia nome do Prisma para coluna real no banco.
21	}	Declara campo, tipo ou configuracao do schema.
22		Linha em branco para separar blocos e melhorar legibilidade.
23	model Product {	Declara entidade do banco que o Prisma usa como modelo.
24	id Int @id @default(autoincrement())	Define chave primaria.
25	name String	Declara campo, tipo ou configuracao do schema.
26	store String	Declara campo, tipo ou configuracao do schema.
27	category String	Declara campo, tipo ou configuracao do schema.
28	currentPrice Float @map("currentPrice")	Mapeia nome do Prisma para coluna real no banco.
29	previousPrice Float- @map("previousPrice")	Mapeia nome do Prisma para coluna real no banco.
30	emoji String	Declara campo, tipo ou configuracao do schema.

Linha	Codigo	Explicacao
31	externalId String- @unique @map("externalId")	Impede duplicidade para este campo.
32	updatedAt DateTime @updatedAt @map("updatedAt")	Mapeia nome do Prisma para coluna real no banco.
33	alerts Alert[]	Declara campo, tipo ou configuracao do schema.
34	priceHistory PriceHistory[]	Declara campo, tipo ou configuracao do schema.
35		Linha em branco para separar blocos e melhorar legibilidade.
36	@@map("products")	Mapeia nome do Prisma para coluna real no banco.
37	}	Declara campo, tipo ou configuracao do schema.
38		Linha em branco para separar blocos e melhorar legibilidade.
39	model Alert {	Declara entidade do banco que o Prisma usa como modelo.
40	id Int @id @default(autoincrement())	Define chave primaria.
41	userId Int @map("userid")	Mapeia nome do Prisma para coluna real no banco.
42	productId Int @map("productid")	Mapeia nome do Prisma para coluna real no banco.
43	targetPrice Float @map("targetPrice")	Mapeia nome do Prisma para coluna real no banco.
44	isActive Boolean @default(true) @map("isActive")	Mapeia nome do Prisma para coluna real no banco.
45	createdAt DateTime @default(now()) @map("createdAt")	Mapeia nome do Prisma para coluna real no banco.
46		Linha em branco para separar blocos e melhorar legibilidade.
47	user User @relation(fields: [userId], references: [id])	Define relacionamento/chave estrangeira.
48	product Product @relation(fields: [productId], references: [id])	Define relacionamento/chave estrangeira.
49		Linha em branco para separar blocos e melhorar legibilidade.
50	@@map("alerts")	Mapeia nome do Prisma para coluna real no banco.
51	}	Declara campo, tipo ou configuracao do schema.
52		Linha em branco para separar blocos e melhorar legibilidade.
53	model PriceHistory {	Declara entidade do banco que o Prisma usa como modelo.
54	id Int @id @default(autoincrement())	Define chave primaria.
55	productId Int @map("productId")	Mapeia nome do Prisma para coluna real no banco.
56	price Float	Declara campo, tipo ou configuracao do schema.
57	createdAt DateTime @default(now()) @map("createdAt")	Mapeia nome do Prisma para coluna real no banco.
58		Linha em branco para separar blocos e melhorar legibilidade.
59	product Product @relation(fields: [productId], references: [id])	Define relacionamento/chave estrangeira.
60		Linha em branco para separar blocos e melhorar legibilidade.
61	@@map("price_history")	Mapeia nome do Prisma para coluna real no banco.
62	}	Declara campo, tipo ou configuracao do schema.

prisma/migrations/20260614142017_init_mobile/migration.sql

SQL real da migracao inicial no MySQL.

Linha	Codigo	Explicacao
1	-- CreateTable	Comando SQL gerado pela migracao.
2	CREATE TABLE `users` (	Cria uma tabela fisica no MySQL.
3	`id` INTEGER NOT NULL AUTO_INCREMENT,	Valor numerico gerado automaticamente.
4	`name` VARCHAR(191) NOT NULL,	Comando SQL gerado pela migracao.
5	`email` VARCHAR(191) NOT NULL,	Comando SQL gerado pela migracao.
6	`password_hash` VARCHAR(191) NOT NULL,	Comando SQL gerado pela migracao.
7	`createdAt` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),	Comando SQL gerado pela migracao.
8		Linha em branco para separar blocos e melhorar legibilidade.
9	UNIQUE INDEX `users_email_key` (`email`),	Cria indice unico.
10	PRIMARY KEY (`id`)	Define chave primaria.
11	) DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;	Comando SQL gerado pela migracao.
12		Linha em branco para separar blocos e melhorar legibilidade.
13	-- CreateTable	Comando SQL gerado pela migracao.
14	CREATE TABLE `products` (	Cria uma tabela fisica no MySQL.
15	`id` INTEGER NOT NULL AUTO_INCREMENT,	Valor numerico gerado automaticamente.
16	`name` VARCHAR(191) NOT NULL,	Comando SQL gerado pela migracao.
17	`store` VARCHAR(191) NOT NULL,	Comando SQL gerado pela migracao.
18	`category` VARCHAR(191) NOT NULL,	Comando SQL gerado pela migracao.
19	`currentPrice` DOUBLE NOT NULL,	Comando SQL gerado pela migracao.
20	`previousPrice` DOUBLE NULL,	Comando SQL gerado pela migracao.
21	`emoji` VARCHAR(191) NOT NULL,	Comando SQL gerado pela migracao.
22	`externalId` VARCHAR(191) NULL,	Comando SQL gerado pela migracao.
23	`updatedAt` DATETIME(3) NOT NULL,	Comando SQL gerado pela migracao.
24		Linha em branco para separar blocos e melhorar legibilidade.
25	UNIQUE INDEX `products_externalId_key` (`externalId`),	Cria indice unico.
26	PRIMARY KEY (`id`)	Define chave primaria.
27	) DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;	Comando SQL gerado pela migracao.
28		Linha em branco para separar blocos e melhorar legibilidade.
29	-- CreateTable	Comando SQL gerado pela migracao.
30	CREATE TABLE `alerts` (	Cria uma tabela fisica no MySQL.

Linha	Codigo	Explicacao
31	<code>`id` INTEGER NOT NULL AUTO_INCREMENT,</code>	Valor numerico gerado automaticamente.
32	<code>`userid` INTEGER NOT NULL,</code>	Comando SQL gerado pela migracao.
33	<code>`productid` INTEGER NOT NULL,</code>	Comando SQL gerado pela migracao.
34	<code>`targetPrice` DOUBLE NOT NULL,</code>	Comando SQL gerado pela migracao.
35	<code>`isActive` BOOLEAN NOT NULL DEFAULT true,</code>	Comando SQL gerado pela migracao.
36	<code>`createdAt` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),</code>	Comando SQL gerado pela migracao.
37		Linha em branco para separar blocos e melhorar legibilidade.
38	<code>PRIMARY KEY (`id`)</code>	Define chave primaria.
39	<code>) DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;</code>	Comando SQL gerado pela migracao.
40		Linha em branco para separar blocos e melhorar legibilidade.
41	<code>-- CreateTable</code>	Comando SQL gerado pela migracao.
42	<code>CREATE TABLE `price_history` (</code>	Cria uma tabela fisica no MySQL.
43	<code>`id` INTEGER NOT NULL AUTO_INCREMENT,</code>	Valor numerico gerado automaticamente.
44	<code>`productId` INTEGER NOT NULL,</code>	Comando SQL gerado pela migracao.
45	<code>`price` DOUBLE NOT NULL,</code>	Comando SQL gerado pela migracao.
46	<code>`createdAt` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),</code>	Comando SQL gerado pela migracao.
47		Linha em branco para separar blocos e melhorar legibilidade.
48	<code>PRIMARY KEY (`id`)</code>	Define chave primaria.
49	<code>) DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;</code>	Comando SQL gerado pela migracao.
50		Linha em branco para separar blocos e melhorar legibilidade.
51	<code>-- AddForeignKey</code>	Comando SQL gerado pela migracao.
52	<code>ALTER TABLE `alerts` ADD CONSTRAINT `alerts_userid_fkey` FOREIGN KEY (`userid`) REFERENCES `users` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE;</code>	Cria relacionamento entre tabelas.
53		Linha em branco para separar blocos e melhorar legibilidade.
54	<code>-- AddForeignKey</code>	Comando SQL gerado pela migracao.
55	<code>ALTER TABLE `alerts` ADD CONSTRAINT `alerts_productid_fkey` FOREIGN KEY (`productid`) REFERENCES `products` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE;</code>	Cria relacionamento entre tabelas.
56		Linha em branco para separar blocos e melhorar legibilidade.
57	<code>-- AddForeignKey</code>	Comando SQL gerado pela migracao.
58	<code>ALTER TABLE `price_history` ADD CONSTRAINT `price_history_productId_fkey` FOREIGN KEY (`productId`) REFERENCES `products` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE;</code>	Cria relacionamento entre tabelas.

src/app.ts

Monta a aplicacao Express com middlewares, rotas e tratamento de erro.

Linha	Codigo	Explicacao
1	import express from "express";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	import { corsConfig } from "../config/cors";	Importa dependencia, tipo ou modulo usado neste arquivo.
4		Linha em branco para separar blocos e melhorar legibilidade.
5	import { router } from "../routes";	Importa dependencia, tipo ou modulo usado neste arquivo.
6		Linha em branco para separar blocos e melhorar legibilidade.
7	import { errorHandler } from "../errors/errorHandler";	Importa dependencia, tipo ou modulo usado neste arquivo.
8		Linha em branco para separar blocos e melhorar legibilidade.
9	const app = express();	Compos a estrutura, regra, tipo ou resposta desse arquivo.
10		Linha em branco para separar blocos e melhorar legibilidade.
11	app.use(corsConfig);	Registra middleware/rotas no Express.
12		Linha em branco para separar blocos e melhorar legibilidade.
13	app.use(express.json());	Registra middleware/rotas no Express.
14		Linha em branco para separar blocos e melhorar legibilidade.
15	app.use(router);	Registra middleware/rotas no Express.
16		Linha em branco para separar blocos e melhorar legibilidade.
17	app.use(errorHandler);	Registra middleware/rotas no Express.
18		Linha em branco para separar blocos e melhorar legibilidade.
19	export { app };	Compos a estrutura, regra, tipo ou resposta desse arquivo.

src/server.ts

Inicializa o servidor HTTP na porta configurada.

Linha	Codigo	Explicacao
1	import { app } from './app';	Importa dependencia, tipo ou modulo usado neste arquivo.
2	import { env } from './config/env';	Importa dependencia, tipo ou modulo usado neste arquivo.
3		Linha em branco para separar blocos e melhorar legibilidade.
4	app.listen(env.PORT, () => {	Inicia servidor na porta configurada.
5	console.log(`Server running on port \${env.PORT}`);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
6	});	Fecha bloco, objeto ou chamada anterior.

src/config/env.ts

Centraliza leitura do .env e valores padrao.

Linha	Codigo	Explicacao
1	import dotenv from "dotenv";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	dotenv.config();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
4		Linha em branco para separar blocos e melhorar legibilidade.
5	const PORT = process.env.PORT - Number(process.env.PORT) : 300;	Le configuracao de variavel de ambiente.
6	const DATABASE_URL = process.env.DATABASE_URL;	Le configuracao de variavel de ambiente.
7		Linha em branco para separar blocos e melhorar legibilidade.
8	if (!DATABASE_URL) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9	throw new Error("DATABASE_URL não definida no arquivo .env");	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	}	Fecha bloco, objeto ou chamada anterior.
11		Linha em branco para separar blocos e melhorar legibilidade.
12	export const env = {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
13	PORT,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
14	DATABASE_URL,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
15	JWT_SECRET: process.env.JWT_SECRET!,	Le configuracao de variavel de ambiente.
16	EXTERNAL_PRODUCTS_PROVIDER:	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
17	process.env.EXTERNAL_PRODUCTS_PROVIDER [carrinho] "mercadolivre",	Le configuracao de variavel de ambiente.
18	MERCADO_LIVRE_API_BASE_URL:	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
19	process.env.MERCADO_LIVRE_API_BASE_URL [carrinho] "https://api.mercadolivre.com",	Le configuracao de variavel de ambiente.
20	MERCADO_LIVRE_SITE_ID: process.env.MERCADO_LIVRE_SITE_ID [carrinho] "MLB",	Le configuracao de variavel de ambiente.
21	};	Fecha bloco, objeto ou chamada anterior.

src/config/cors.ts

Libera acesso CORS para o app/front.

Linha	Codigo	Explicacao
1	import cors from "cors";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	export const corsConfig = cors({	Compos a estrutura, regra, tipo ou resposta desse arquivo.
4	origin: "*",	Compos a estrutura, regra, tipo ou resposta desse arquivo.
5	methods: ["GET", "POST", "PUT", "DELETE"],	Compos a estrutura, regra, tipo ou resposta desse arquivo.
6	});	Fecha bloco, objeto ou chamada anterior.

src/prisma/client.ts

Cria o PrismaClient usado para acessar o banco.

Linha	Codigo	Explicacao
1	import "dotenv/config";	Importa dependencia, tipo ou modulo usado neste arquivo.
2	import { PrismaClient } from "@prisma/client";	Importa dependencia, tipo ou modulo usado neste arquivo.
3		Linha em branco para separar blocos e melhorar legibilidade.
4	export const prisma = new PrismaClient();	Compoee a estrutura, regra, tipo ou resposta desse arquivo.

src/routes/index.ts

Agrupar /auth, /products e /external-products.

Linha	Codigo	Explicacao
1	import { Router } from "express";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	import { authRoutes } from "./auth.routes";	Importa dependencia, tipo ou modulo usado neste arquivo.
4	import productRoutes from "./product.routes";	Importa dependencia, tipo ou modulo usado neste arquivo.
5	import externalProductRoutes from "./external-products.routes";	Importa dependencia, tipo ou modulo usado neste arquivo.
6		Linha em branco para separar blocos e melhorar legibilidade.
7	const router = Router();	Cria roteador Express para agrupar endpoints.
8		Linha em branco para separar blocos e melhorar legibilidade.
9	router.use("/auth", authRoutes);	Conecta um grupo de rotas a um prefixo.
10	router.use("/products", productRoutes);	Conecta um grupo de rotas a um prefixo.
11	router.use("/external-products", externalProductRoutes);	Conecta um grupo de rotas a um prefixo.
12		Linha em branco para separar blocos e melhorar legibilidade.
13	export { router };	Compos a estrutura, regra, tipo ou resposta desse arquivo.

src/routes/auth.routes.ts

Rotas de cadastro e login.

Linha	Codigo	Explicacao
1	import { Router } from "express";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	import { authController } from "../controllers/auth.controller";	Importa dependencia, tipo ou modulo usado neste arquivo.
4	import { validateDto } from "../middleware/validation.middleware";	Importa dependencia, tipo ou modulo usado neste arquivo.
5	import { CreateUserDto } from "../dtos/create-user.dto";	Importa dependencia, tipo ou modulo usado neste arquivo.
6		Linha em branco para separar blocos e melhorar legibilidade.
7	const router = Router();	Cria roteador Express para agrupar endpoints.
8		Linha em branco para separar blocos e melhorar legibilidade.
9	router.post(	Registra rota HTTP POST.
10	"/register",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
11	validateDto(CreateUserDto, "body"),	Aplica validacao antes do controller.
12	authController.register.bind(authController)	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
13	);	Fecha bloco, objeto ou chamada anterior.
14		Linha em branco para separar blocos e melhorar legibilidade.
15	router.post(	Registra rota HTTP POST.
16	"/login",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
17	authController.login.bind(authController)	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
18	);	Fecha bloco, objeto ou chamada anterior.
19		Linha em branco para separar blocos e melhorar legibilidade.
20	export { router as authRoutes };	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

src/routes/product.routes.ts

CRUD de produtos e rotas personalizadas do comparador.

Linha	Codigo	Explicacao
1	import { Router } from "express";	Importa dependencia, tipo ou modulo usado neste arquivo.
2	import { ProductController } from "../controllers/products.controller";	Importa dependencia, tipo ou modulo usado neste arquivo.
3	import { validateDto } from "../middleware/validation.middleware";	Importa dependencia, tipo ou modulo usado neste arquivo.
4	import { ProductListDto } from "../dtos/product-list.dto";	Importa dependencia, tipo ou modulo usado neste arquivo.
5	import {	Importa dependencia, tipo ou modulo usado neste arquivo.
6	CreateProductDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
7	IdParamDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8	PriceHistoryQueryDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9	ProductSearchDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	ProductTrendingDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
11	RecordPriceSnapshotDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
12	UpdateProductDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
13	} from "../dtos/product.dto";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
14		Linha em branco para separar blocos e melhorar legibilidade.
15	const productRoutes = Router();	Cria roteador Express para agrupar endpoints.
16	const productController = new ProductController();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
17		Linha em branco para separar blocos e melhorar legibilidade.
18		Linha em branco para separar blocos e melhorar legibilidade.
19	productRoutes.get(	Registra rota HTTP GET.
20	"/",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
21	validateDto(ProductListDto, "query"),	Aplica validacao antes do controller.
22	productController.list	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
23	);	Fecha bloco, objeto ou chamada anterior.
24		Linha em branco para separar blocos e melhorar legibilidade.
25	productRoutes.get(	Registra rota HTTP GET.
26	"/search",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
27	validateDto(ProductSearchDto, "query"),	Aplica validacao antes do controller.
28	productController.search	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
29	);	Fecha bloco, objeto ou chamada anterior.
30		Linha em branco para separar blocos e melhorar legibilidade.

Linha	Codigo	Explicacao
31	productRoutes.get(	Registra rota HTTP GET.
32	"/trending",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
33	validateDto(ProductTrendingDto, "query"),	Aplica validacao antes do controller.
34	productController.trending	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
35	);	Fecha bloco, objeto ou chamada anterior.
36		Linha em branco para separar blocos e melhorar legibilidade.
37	productRoutes.get(	Registra rota HTTP GET.
38	"/:id",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
39	validateDto(IdParamDto, "params"),	Aplica validacao antes do controller.
40	productController.findById	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
41	);	Fecha bloco, objeto ou chamada anterior.
42		Linha em branco para separar blocos e melhorar legibilidade.
43	productRoutes.get(	Registra rota HTTP GET.
44	"/:id/offers",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
45	validateDto(IdParamDto, "params"),	Aplica validacao antes do controller.
46	productController.findOffers	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
47	);	Fecha bloco, objeto ou chamada anterior.
48		Linha em branco para separar blocos e melhorar legibilidade.
49	productRoutes.get(	Registra rota HTTP GET.
50	"/:id/price-history",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
51	validateDto(IdParamDto, "params"),	Aplica validacao antes do controller.
52	validateDto(PriceHistoryQueryDto, "query"),	Aplica validacao antes do controller.
53	productController.findPriceHistory	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
54	);	Fecha bloco, objeto ou chamada anterior.
55		Linha em branco para separar blocos e melhorar legibilidade.
56	productRoutes.post(	Registra rota HTTP POST.
57	"/:id/price-snapshots",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
58	validateDto(IdParamDto, "params"),	Aplica validacao antes do controller.
59	validateDto(RecordPriceSnapshotDto, "body"),	Aplica validacao antes do controller.
60	productController.recordPriceSnapshot	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
61	);	Fecha bloco, objeto ou chamada anterior.
62		Linha em branco para separar blocos e melhorar legibilidade.
63	productRoutes.post(	Registra rota HTTP POST.

Linha	Codigo	Explicacao
64	"/",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
65	validateDto(CreateProductDto, "body"),	Aplica validacao antes do controller.
66	productController.create	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
67	);	Fecha bloco, objeto ou chamada anterior.
68		Linha em branco para separar blocos e melhorar legibilidade.
69	productRoutes.put(	Registra rota HTTP PUT.
70	("/:id",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
71	validateDto(IdParamDto, "params"),	Aplica validacao antes do controller.
72	validateDto(UpdateProductDto, "body"),	Aplica validacao antes do controller.
73	productController.update	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
74	);	Fecha bloco, objeto ou chamada anterior.
75		Linha em branco para separar blocos e melhorar legibilidade.
76	productRoutes.delete(	Registra rota HTTP DELETE.
77	("/:id",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
78	validateDto(IdParamDto, "params"),	Aplica validacao antes do controller.
79	productController.delete	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
80	);	Fecha bloco, objeto ou chamada anterior.
81		Linha em branco para separar blocos e melhorar legibilidade.
82		Linha em branco para separar blocos e melhorar legibilidade.
83	export default productRoutes;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

src/routes/external-products.routes.ts

Rotas para buscar/importar produtos externos.

Linha	Codigo	Explicacao
1	import { Router } from "express";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	import { ExternalProductsController } from "../controllers/external-products.controller";	Importa dependencia, tipo ou modulo usado neste arquivo.
4	import { validateDto } from "../middleware/validation.middleware";	Importa dependencia, tipo ou modulo usado neste arquivo.
5	import {	Importa dependencia, tipo ou modulo usado neste arquivo.
6	ExternalProductSearchDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
7	ImportExternalProductsDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8	} from "../dtos/external-products.dto";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9		Linha em branco para separar blocos e melhorar legibilidade.
10	const externalProductRoutes = Router();	Cria roteador Express para agrupar endpoints.
11	const externalProductsController = new ExternalProductsController();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
12		Linha em branco para separar blocos e melhorar legibilidade.
13	externalProductRoutes.get(	Registra rota HTTP GET.
14	"/search",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
15	validateDto(ExternalProductSearchDto, "query"),	Aplica validacao antes do controller.
16	externalProductsController.search	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
17	);	Fecha bloco, objeto ou chamada anterior.
18		Linha em branco para separar blocos e melhorar legibilidade.
19	externalProductRoutes.post(	Registra rota HTTP POST.
20	"/import",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
21	validateDto(ImportExternalProductsDto, "body"),	Aplica validacao antes do controller.
22	externalProductsController.importProducts	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
23	);	Fecha bloco, objeto ou chamada anterior.
24		Linha em branco para separar blocos e melhorar legibilidade.
25	export default externalProductRoutes;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

src/controllers/auth.controller.ts

Controller de autenticação.

Linha	Codigo	Explicacao
1	import { Request, Response, NextFunction } from "express";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	import { authService } from "../services/auth.service";	Importa dependencia, tipo ou modulo usado neste arquivo.
4		Linha em branco para separar blocos e melhorar legibilidade.
5	export class AuthController {	Declara classe exportada usada por outras camadas.
6	async register(	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
7	req: Request,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8	res: Response,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9	next: NextFunction	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
11	try {	Inicia bloco protegido contra erros.
12	const result = await authService.register(req.body);	Usa dados do corpo da requisicao.
13		Linha em branco para separar blocos e melhorar legibilidade.
14	return res.status(201).json(result);	Define status HTTP e envia resposta.
15	} catch (error) {	Encaminha erro ao errorHandler.
16	next(error);	Encaminha erro ao errorHandler.
17	}	Fecha bloco, objeto ou chamada anterior.
18	}	Fecha bloco, objeto ou chamada anterior.
19		Linha em branco para separar blocos e melhorar legibilidade.
20	async login(	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
21	req: Request,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
22	res: Response,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
23	next: NextFunction	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
24	) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
25	try {	Inicia bloco protegido contra erros.
26	const result = await authService.login(req.body);	Usa dados do corpo da requisicao.
27		Linha em branco para separar blocos e melhorar legibilidade.
28	return res.status(200).json(result);	Define status HTTP e envia resposta.
29	} catch (error) {	Encaminha erro ao errorHandler.
30	next(error);	Encaminha erro ao errorHandler.

Linha	Codigo	Explicacao
31	}	Fecha bloco, objeto ou chamada anterior.
32	}	Fecha bloco, objeto ou chamada anterior.
33	}	Fecha bloco, objeto ou chamada anterior.
34		Linha em branco para separar blocos e melhorar legibilidade.
35	<code>export const authController = new AuthController();</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

src/controllers/products.controller.ts

Controller de produtos.

Linha	Codigo	Explicacao
1	import { Request, Response, NextFunction } from "express";	Importa dependencia, tipo ou modulo usado neste arquivo.
2	import { ProductService } from "../services/product.service";	Importa dependencia, tipo ou modulo usado neste arquivo.
3	import { ProductListDto } from "../dtos/product-list.dto";	Importa dependencia, tipo ou modulo usado neste arquivo.
4	import {	Importa dependencia, tipo ou modulo usado neste arquivo.
5	CreateProductDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
6	PriceHistoryQueryDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
7	ProductSearchDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8	ProductTrendingDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9	RecordPriceSnapshotDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	UpdateProductDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
11	} from "../dtos/product.dto";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
12		Linha em branco para separar blocos e melhorar legibilidade.
13	export class ProductController {	Declara classe exportada usada por outras camadas.
14	private productService: ProductService;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
15		Linha em branco para separar blocos e melhorar legibilidade.
16	constructor() {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
17	this.productService = new ProductService();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
18	}	Fecha bloco, objeto ou chamada anterior.
19		Linha em branco para separar blocos e melhorar legibilidade.
20	list = async (req: Request, res: Response, next: NextFunction) => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
21	try {	Inicia bloco protegido contra erros.
22	const query = req.query as unknown as ProductListDto;	Usa parametros de query string.
23		Linha em branco para separar blocos e melhorar legibilidade.
24	const products = await this.productService.list(query);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
25		Linha em branco para separar blocos e melhorar legibilidade.
26	return res.status(200).json(products);	Define status HTTP e envia resposta.
27	} catch (error) {	Encaminha erro ao errorHandler.
28	next(error);	Encaminha erro ao errorHandler.
29	}	Fecha bloco, objeto ou chamada anterior.
30	}	Fecha bloco, objeto ou chamada anterior.

Linha	Codigo	Explicacao
31		Linha em branco para separar blocos e melhorar legibilidade.
32	search = async (req: Request, res: Response, next: NextFunction) => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
33	try {	Inicia bloco protegido contra erros.
34	const query = req.query as unknown as ProductSearchDto;	Usa parametros de query string.
35	const products = await this.productService.search(query);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
36		Linha em branco para separar blocos e melhorar legibilidade.
37	return res.status(200).json(products);	Define status HTTP e envia resposta.
38	} catch (error) {	Encaminha erro ao errorHandler.
39	next(error);	Encaminha erro ao errorHandler.
40	}	Fecha bloco, objeto ou chamada anterior.
41	}	Fecha bloco, objeto ou chamada anterior.
42		Linha em branco para separar blocos e melhorar legibilidade.
43	trending = async (req: Request, res: Response, next: NextFunction) => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
44	try {	Inicia bloco protegido contra erros.
45	const query = req.query as unknown as ProductTrendingDto;	Usa parametros de query string.
46	const products = await this.productService.trending(query);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
47		Linha em branco para separar blocos e melhorar legibilidade.
48	return res.status(200).json(products);	Define status HTTP e envia resposta.
49	} catch (error) {	Encaminha erro ao errorHandler.
50	next(error);	Encaminha erro ao errorHandler.
51	}	Fecha bloco, objeto ou chamada anterior.
52	}	Fecha bloco, objeto ou chamada anterior.
53		Linha em branco para separar blocos e melhorar legibilidade.
54	findById = async (req: Request, res: Response, next: NextFunction) => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
55	try {	Inicia bloco protegido contra erros.
56	const id = Number(req.params.id);	Usa parametros dinamicos da URL.
57	const product = await this.productService.findById(id);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
58		Linha em branco para separar blocos e melhorar legibilidade.
59	return res.status(200).json(product);	Define status HTTP e envia resposta.
60	} catch (error) {	Encaminha erro ao errorHandler.
61	next(error);	Encaminha erro ao errorHandler.
62	}	Fecha bloco, objeto ou chamada anterior.
63	}	Fecha bloco, objeto ou chamada anterior.

Linha	Codigo	Explicacao
64		Linha em branco para separar blocos e melhorar legibilidade.
65	findOffers = async (req: Request, res: Response, next: NextFunction) => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
66	try {	Inicia bloco protegido contra erros.
67	const id = Number(req.params.id);	Usa parametros dinamicos da URL.
68	const offers = await this.productService.findOffers(id);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
69		Linha em branco para separar blocos e melhorar legibilidade.
70	return res.status(200).json(offers);	Define status HTTP e envia resposta.
71	} catch (error) {	Encaminha erro ao errorHandler.
72	next(error);	Encaminha erro ao errorHandler.
73	}	Fecha bloco, objeto ou chamada anterior.
74	}	Fecha bloco, objeto ou chamada anterior.
75		Linha em branco para separar blocos e melhorar legibilidade.
76	findPriceHistory = async (req: Request, res: Response, next: NextFunction) => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
77	try {	Inicia bloco protegido contra erros.
78	const id = Number(req.params.id);	Usa parametros dinamicos da URL.
79	const query = req.query as unknown as PriceHistoryQueryDto;	Usa parametros de query string.
80	const history = await this.productService.findPriceHistory(id, query);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
81		Linha em branco para separar blocos e melhorar legibilidade.
82	return res.status(200).json(history);	Define status HTTP e envia resposta.
83	} catch (error) {	Encaminha erro ao errorHandler.
84	next(error);	Encaminha erro ao errorHandler.
85	}	Fecha bloco, objeto ou chamada anterior.
86	}	Fecha bloco, objeto ou chamada anterior.
87		Linha em branco para separar blocos e melhorar legibilidade.
88	recordPriceSnapshot = async (	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
89	req: Request<{ id: string }, {}, RecordPriceSnapshotDto>,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
90	res: Response,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
91	next: NextFunction	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
92	) => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
93	try {	Inicia bloco protegido contra erros.
94	const id = Number(req.params.id);	Usa parametros dinamicos da URL.
95	const product = await this.productService.recordPriceSnapshot(id, req.body);	Usa dados do corpo da requisicao.
96		Linha em branco para separar blocos e melhorar legibilidade.

Linha	Codigo	Explicacao
97	<code>return res.status(200).json(product);</code>	Define status HTTP e envia resposta.
98	<code>} catch (error) {</code>	Encaminha erro ao errorHandler.
99	<code>next(error);</code>	Encaminha erro ao errorHandler.
100	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
101	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
102		Linha em branco para separar blocos e melhorar legibilidade.
103	<code>create = async (</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
104	<code>req: Request<{}, {}>, CreateProductDto,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
105	<code>res: Response,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
106	<code>next: NextFunction</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
107	<code>) => {</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
108	<code>try {</code>	Inicia bloco protegido contra erros.
109	<code>const newProduct = await this.productService.create(req.body);</code>	Usa dados do corpo da requisicao.
110		Linha em branco para separar blocos e melhorar legibilidade.
111	<code>return res.status(201).json(newProduct);</code>	Define status HTTP e envia resposta.
112	<code>} catch(error) {</code>	Encaminha erro ao errorHandler.
113	<code>next(error);</code>	Encaminha erro ao errorHandler.
114	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
115	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
116		Linha em branco para separar blocos e melhorar legibilidade.
117	<code>update = async (</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
118	<code>req: Request<{ id: string }>, {}, UpdateProductDto,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
119	<code>res: Response,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
120	<code>next: NextFunction</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
121	<code>) => {</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
122	<code>try {</code>	Inicia bloco protegido contra erros.
123	<code>const id = Number(req.params.id);</code>	Usa parametros dinamicos da URL.
124	<code>const updateProduct = await this.productService.update(id, req.body);</code>	Usa dados do corpo da requisicao.
125		Linha em branco para separar blocos e melhorar legibilidade.
126	<code>return res.status(200).json(updateProduct);</code>	Define status HTTP e envia resposta.
127	<code>} catch(error) {</code>	Encaminha erro ao errorHandler.
128	<code>next(error);</code>	Encaminha erro ao errorHandler.
129	<code>}</code>	Fecha bloco, objeto ou chamada anterior.

Linha	Codigo	Explicacao
130	}	Fecha bloco, objeto ou chamada anterior.
131		Linha em branco para separar blocos e melhorar legibilidade.
132	delete = async (req: Request, res: Response, next: NextFunction) => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
133	try {	Inicia bloco protegido contra erros.
134	const id = Number(req.params.id);	Usa parametros dinamicos da URL.
135	await this.productService.delete(id);	Registra rota HTTP DELETE.
136		Linha em branco para separar blocos e melhorar legibilidade.
137	return res.status(204).send();	Define status HTTP e envia resposta.
138	} catch(error) {	Encaminha erro ao errorHandler.
139	next(error);	Encaminha erro ao errorHandler.
140	}	Fecha bloco, objeto ou chamada anterior.
141	}	Fecha bloco, objeto ou chamada anterior.
142		Linha em branco para separar blocos e melhorar legibilidade.
143	}	Fecha bloco, objeto ou chamada anterior.
144		Linha em branco para separar blocos e melhorar legibilidade.
145	export const productController = new ProductController();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

src/controllers/external-products.controller.ts

Controller da integracao externa.

Linha	Codigo	Explicacao
1	import { NextFunction, Request, Response } from "express";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	import { ExternalProductService } from "../services/external-product.service";	Importa dependencia, tipo ou modulo usado neste arquivo.
4	import {	Importa dependencia, tipo ou modulo usado neste arquivo.
5	ExternalProductSearchDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
6	ImportExternalProductsDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
7	} from "../dtos/external-products.dto";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8		Linha em branco para separar blocos e melhorar legibilidade.
9	export class ExternalProductsController {	Declara classe exportada usada por outras camadas.
10	private externalProductService: ExternalProductService;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
11		Linha em branco para separar blocos e melhorar legibilidade.
12	constructor() {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
13	this.externalProductService = new ExternalProductService();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
14	}	Fecha bloco, objeto ou chamada anterior.
15		Linha em branco para separar blocos e melhorar legibilidade.
16	search = async (req: Request, res: Response, next: NextFunction) => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
17	try {	Inicia bloco protegido contra erros.
18	const query = req.query as unknown as ExternalProductSearchDto;	Usa parametros de query string.
19	const products = await this.externalProductService.search(query);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
20		Linha em branco para separar blocos e melhorar legibilidade.
21	return res.status(200).json(products);	Define status HTTP e envia resposta.
22	} catch (error) {	Encaminha erro ao errorHandler.
23	next(error);	Encaminha erro ao errorHandler.
24	}	Fecha bloco, objeto ou chamada anterior.
25	}	Fecha bloco, objeto ou chamada anterior.
26		Linha em branco para separar blocos e melhorar legibilidade.
27	importProducts = async (	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
28	req: Request<{}>, {}, ImportExternalProductsDto>,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
29	res: Response,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
30	next: NextFunction	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Linha	Codigo	Explicacao
31	) => {	Compos a estrutura, regra, tipo ou resposta desse arquivo.
32	try {	Inicia bloco protegido contra erros.
33	const result = await this.externalProductService.importProducts(req.body);	Usa dados do corpo da requisicao.
34		Linha em branco para separar blocos e melhorar legibilidade.
35	return res.status(201).json(result);	Define status HTTP e envia resposta.
36	} catch (error) {	Encaminha erro ao errorHandler.
37	next(error);	Encaminha erro ao errorHandler.
38	}	Fecha bloco, objeto ou chamada anterior.
39	}	Fecha bloco, objeto ou chamada anterior.
40	}	Fecha bloco, objeto ou chamada anterior.

src/controllers/alert.controller.ts

Reservado para alertas; atualmente vazio.

Este arquivo esta vazio no estado atual do projeto. Ele existe como ponto reservado para implementacao futura.

src/services/auth.service.ts

Regra de cadastro/login, hash e JWT.

Linha	Codigo	Explicacao
1	import { prisma } from "../prisma/client";	Importa dependencia, tipo ou modulo usado neste arquivo.
2	import { CreateUserDto } from "../dtos/create-user.dto";	Importa dependencia, tipo ou modulo usado neste arquivo.
3	import { LoginDto } from "../dtos/login.dto";	Importa dependencia, tipo ou modulo usado neste arquivo.
4		Linha em branco para separar blocos e melhorar legibilidade.
5	import { hashPassword, comparePassword } from "../utils/bcrypt";	Importa dependencia, tipo ou modulo usado neste arquivo.
6	import { generateToken } from "../utils/jwt";	Importa dependencia, tipo ou modulo usado neste arquivo.
7		Linha em branco para separar blocos e melhorar legibilidade.
8	import { AppError } from "../errors/AppError";	Importa dependencia, tipo ou modulo usado neste arquivo.
9		Linha em branco para separar blocos e melhorar legibilidade.
10	export class AuthService {	Declara classe exportada usada por outras camadas.
11	async register(data: CreateUserDto) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
12	const { name, email, password } = data;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
13		Linha em branco para separar blocos e melhorar legibilidade.
14	if (!name !email !password) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
15	throw new AppError("All fields are required", 400);	Gera erro controlado com status HTTP.
16	}	Fecha bloco, objeto ou chamada anterior.
17		Linha em branco para separar blocos e melhorar legibilidade.
18	const existingUser = await prisma.user.findUnique({	Acessa o banco pelo Prisma Client.
19	where: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
20	email,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
21	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
22	});	Fecha bloco, objeto ou chamada anterior.
23		Linha em branco para separar blocos e melhorar legibilidade.
24	if (existingUser) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
25	throw new AppError("Email already registered", 409);	Gera erro controlado com status HTTP.
26	}	Fecha bloco, objeto ou chamada anterior.
27		Linha em branco para separar blocos e melhorar legibilidade.
28	const passwordHash = await hashPassword(password);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
29		Linha em branco para separar blocos e melhorar legibilidade.
30	const user = await prisma.user.create({	Acessa o banco pelo Prisma Client.

Linha	Codigo	Explicacao
31	data: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
32	name,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
33	email,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
34	passwordHash,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
35	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
36	});	Fecha bloco, objeto ou chamada anterior.
37		Linha em branco para separar blocos e melhorar legibilidade.
38	const token = generateToken(user.id);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
39		Linha em branco para separar blocos e melhorar legibilidade.
40	return {	Inicia objeto de resposta/retorno.
41	user: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
42	id: user.id,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
43	name: user.name,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
44	email: user.email,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
45	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
46	token,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
47	};	Fecha bloco, objeto ou chamada anterior.
48	}	Fecha bloco, objeto ou chamada anterior.
49		Linha em branco para separar blocos e melhorar legibilidade.
50	async login(data: LoginDto) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
51	const { login, password } = data;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
52		Linha em branco para separar blocos e melhorar legibilidade.
53	if (!login !password) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
54	throw new AppError("Login and password are required", 400);	Gera erro controlado com status HTTP.
55	}	Fecha bloco, objeto ou chamada anterior.
56		Linha em branco para separar blocos e melhorar legibilidade.
57	const user = await prisma.user.findFirst({	Acessa o banco pelo Prisma Client.
58	where: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
59	OR: [	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
60	{ email: login },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
61	{ name: login }	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
62	]	Fecha bloco, objeto ou chamada anterior.
63	}	Fecha bloco, objeto ou chamada anterior.

Linha	Codigo	Explicacao
64	});	Fecha bloco, objeto ou chamada anterior.
65		Linha em branco para separar blocos e melhorar legibilidade.
66	if (!user) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
67	throw new AppError("Invalid credentials", 401);	Gera erro controlado com status HTTP.
68	}	Fecha bloco, objeto ou chamada anterior.
69		Linha em branco para separar blocos e melhorar legibilidade.
70	const passwordValid = await comparePassword(	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
71	password,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
72	user.passwordHash	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
73	);	Fecha bloco, objeto ou chamada anterior.
74		Linha em branco para separar blocos e melhorar legibilidade.
75	if (!passwordValid) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
76	throw new AppError("Invalid credentials", 401);	Gera erro controlado com status HTTP.
77	}	Fecha bloco, objeto ou chamada anterior.
78		Linha em branco para separar blocos e melhorar legibilidade.
79	const token = generateToken(user.id);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
80		Linha em branco para separar blocos e melhorar legibilidade.
81	return {	Inicia objeto de resposta/retorno.
82	user: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
83	id: user.id,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
84	name: user.name,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
85	email: user.email,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
86	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
87	token,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
88	};	Fecha bloco, objeto ou chamada anterior.
89	}	Fecha bloco, objeto ou chamada anterior.
90	}	Fecha bloco, objeto ou chamada anterior.
91		Linha em branco para separar blocos e melhorar legibilidade.
92		Linha em branco para separar blocos e melhorar legibilidade.
93	export const authService = new AuthService();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

src/services/product.service.ts

Regra principal de produtos, ofertas, historico e snapshots.

Linha	Codigo	Explicacao
1	import { Prisma, Product } from "@prisma/client";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	import { prisma } from "../prisma/client";	Importa dependencia, tipo ou modulo usado neste arquivo.
4	import { AppError } from "../errors/AppError";	Importa dependencia, tipo ou modulo usado neste arquivo.
5	import { ProductListDto } from "../dtos/product-list.dto";	Importa dependencia, tipo ou modulo usado neste arquivo.
6	import {	Importa dependencia, tipo ou modulo usado neste arquivo.
7	CreateProductDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8	PriceHistoryQueryDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9	ProductSearchDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	ProductTrendingDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
11	RecordPriceSnapshotDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
12	UpdateProductDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
13	} from "../dtos/product.dto";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
14		Linha em branco para separar blocos e melhorar legibilidade.
15	export interface ProductListQuery {	Define contrato de dados TypeScript.
16	name-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
17	store-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
18	category-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
19	minPrice-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
20	maxPrice-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
21	}	Fecha bloco, objeto ou chamada anterior.
22		Linha em branco para separar blocos e melhorar legibilidade.
23	export class ProductService {	Declara classe exportada usada por outras camadas.
24	private calcularVariation(current: number, previous: number null): number {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
25	if (!previous) return 0;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
26	return ((current - previous) / previous) * 100;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
27	}	Fecha bloco, objeto ou chamada anterior.
28		Linha em branco para separar blocos e melhorar legibilidade.
29	private normalizeProductKey(product: Pick<Product, "name" "category">): string {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
30	return `\${product.name.trim().toLowerCase()}::\${product.category.trim().toLowerCase()}`;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Linha	Codigo	Explicacao
31	}	Fecha bloco, objeto ou chamada anterior.
32		Linha em branco para separar blocos e melhorar legibilidade.
33	private getStoreInitials(store: string): string {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
34	return store	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
35	.split(/\s+/)	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
36	.map((word) => word[0])	Transforma itens de uma lista.
37	.join("")	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
38	.slice(0, 3)	Limita quantidade de resultados.
39	.toUpperCase();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
40	}	Fecha bloco, objeto ou chamada anterior.
41		Linha em branco para separar blocos e melhorar legibilidade.
42	private groupProductsByCatalog(products: Product[]): Product[][] {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
43	const groups = new Map<string, Product[]>();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
44		Linha em branco para separar blocos e melhorar legibilidade.
45	for (const product of products) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
46	const key = this.normalizeProductKey(product);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
47	const currentGroup = groups.get(key) [carrinho] [];	Registra rota HTTP GET.
48	currentGroup.push(product);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
49	groups.set(key, currentGroup);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
50	}	Fecha bloco, objeto ou chamada anterior.
51		Linha em branco para separar blocos e melhorar legibilidade.
52	return Array.from(groups.values());	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
53	}	Fecha bloco, objeto ou chamada anterior.
54		Linha em branco para separar blocos e melhorar legibilidade.
55	private getCheapestProduct(products: Product[]): Product {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
56	return products.reduce((cheapest, product) =>	Calcula valor agregado, como menor preco ou media.
57	product.currentPrice < cheapest.currentPrice - product : cheapest	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
58	);	Fecha bloco, objeto ou chamada anterior.
59	}	Fecha bloco, objeto ou chamada anterior.
60		Linha em branco para separar blocos e melhorar legibilidade.
61	private toProductSummary(products: Product[]) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
62	const cheapest = this.getCheapestProduct(products);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
63	const prices = products.map((product) => product.currentPrice);	Transforma itens de uma lista.

Linha	Codigo	Explicacao
64	<code>const stores = products.map((product) => product.store);</code>	Transforma itens de uma lista.
65	<code>const variationPercentage = this.calcularVariation(</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
66	<code>cheapest.currentPrice,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
67	<code>cheapest.previousPrice</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
68	<code>);</code>	Fecha bloco, objeto ou chamada anterior.
69		Linha em branco para separar blocos e melhorar legibilidade.
70	<code>return {</code>	Inicia objeto de resposta/retorno.
71	<code>id: cheapest.id,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
72	<code>name: cheapest.name,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
73	<code>store: cheapest.store,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
74	<code>category: cheapest.category,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
75	<code>currentPrice: cheapest.currentPrice,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
76	<code>previousPrice: cheapest.previousPrice,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
77	<code>emoji: cheapest.emoji,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
78	<code>externalId: cheapest.externalId,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
79	<code>updatedAt: cheapest.updatedAt,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
80	<code>variationPercentage,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
81	<code>offersCount: products.length,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
82	<code>stores,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
83	<code>lowestPrice: Math.min(...prices),</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
84	<code>highestPrice: Math.max(...prices),</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
85	<code>averagePrice: prices.reduce((sum, price) => sum + price, 0) / prices.length,</code>	Calcula valor agregado, como menor preco ou media.
86	<code>};</code>	Fecha bloco, objeto ou chamada anterior.
87	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
88		Linha em branco para separar blocos e melhorar legibilidade.
89	<code>private toOffer(product: Product) {</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
90	<code>return {</code>	Inicia objeto de resposta/retorno.
91	<code>id: product.id,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
92	<code>storeName: product.store,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
93	<code>storeInitials: this.getStoreInitials(product.store),</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
94	<code>price: product.currentPrice,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
95	<code>previousPrice: product.previousPrice,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
96	<code>variationPercentage: this.calcularVariation(</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Linha	Codigo	Explicacao
97	product.currentPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
98	product.previousPrice	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
99	),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
100	externalId: product.externalId,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
101	updatedAt: product.updatedAt,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
102	};	Fecha bloco, objeto ou chamada anterior.
103	}	Fecha bloco, objeto ou chamada anterior.
104		Linha em branco para separar blocos e melhorar legibilidade.
105	private buildSearchWhere(query: ProductSearchDto): Prisma.ProductWhereInput {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
106	const { q, store, category, minPrice, maxPrice } = query;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
107		Linha em branco para separar blocos e melhorar legibilidade.
108	return {	Inicia objeto de resposta/retorno.
109	...(q && {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
110	OR: [	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
111	{ name: { contains: q } },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
112	{ store: { contains: q } },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
113	{ category: { contains: q } },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
114	],	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
115	)),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
116	...(store && { store: { contains: store } })),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
117	...(category && { category: { contains: category } })),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
118	...((minPrice !== undefined maxPrice !== undefined) && {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
119	currentPrice: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
120	...(minPrice !== undefined && { gte: minPrice })),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
121	...(maxPrice !== undefined && { lte: maxPrice })),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
122	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
123	)),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
124	};	Fecha bloco, objeto ou chamada anterior.
125	}	Fecha bloco, objeto ou chamada anterior.
126		Linha em branco para separar blocos e melhorar legibilidade.
127		Linha em branco para separar blocos e melhorar legibilidade.
128	async list(query: ProductListDto) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
129	const { name, store, category, minPrice, maxPrice } = query;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Linha	Codigo	Explicacao
130		Linha em branco para separar blocos e melhorar legibilidade.
131	const where: Prisma.ProductWhereInput = {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
132	...(name && { name: { contains: name } })),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
133	...(store && { store: { contains: store } })),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
134	...(category && { category: { contains: category } })),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
135	...((minPrice !== undefined maxPrice !== undefined) && {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
136	currentPrice: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
137	...(minPrice !== undefined && { gte: minPrice })),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
138	...(maxPrice !== undefined && { lte: maxPrice })),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
139	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
140	}},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
141	};	Fecha bloco, objeto ou chamada anterior.
142		Linha em branco para separar blocos e melhorar legibilidade.
143	const products = await prisma.product.findMany({	Acessa o banco pelo Prisma Client.
144	where,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
145	orderBy: { updatedAt: "desc" },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
146	});	Fecha bloco, objeto ou chamada anterior.
147		Linha em branco para separar blocos e melhorar legibilidade.
148	return products.map((product) => ({	Transforma itens de uma lista.
149	...product,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
150	variationPercentage: this.calcularVariation(	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
151	product.currentPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
152	product.previousPrice	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
153	)	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
154	}}));	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
155	}	Fecha bloco, objeto ou chamada anterior.
156		Linha em branco para separar blocos e melhorar legibilidade.
157	async search(query: ProductSearchDto) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
158	const products = await prisma.product.findMany({	Acessa o banco pelo Prisma Client.
159	where: this.buildSearchWhere(query),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
160	orderBy: [	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
161	{ currentPrice: "asc" },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
162	{ updatedAt: "desc" },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Linha	Codigo	Explicacao
163	],	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
164	});	Fecha bloco, objeto ou chamada anterior.
165		Linha em branco para separar blocos e melhorar legibilidade.
166	return this.groupProductsByCatalog(products)	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
167	.map((group) => this.toProductSummary(group))	Transforma itens de uma lista.
168	.sort((a, b) => a.currentPrice - b.currentPrice)	Ordena itens antes de responder.
169	.slice(0, query.limit [carrinho] 20);	Limita quantidade de resultados.
170	}	Fecha bloco, objeto ou chamada anterior.
171		Linha em branco para separar blocos e melhorar legibilidade.
172	async trending(query: ProductTrendingDto) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
173	const products = await prisma.product.findMany({	Acessa o banco pelo Prisma Client.
174	orderBy: { updatedAt: "desc" },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
175	take: 200,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
176	});	Fecha bloco, objeto ou chamada anterior.
177		Linha em branco para separar blocos e melhorar legibilidade.
178	return this.groupProductsByCatalog(products)	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
179	.map((group) => {	Transforma itens de uma lista.
180	const summary = this.toProductSummary(group);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
181	const isPriceDown = summary.variationPercentage < 0;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
182		Linha em branco para separar blocos e melhorar legibilidade.
183	return {	Inicia objeto de resposta/retorno.
184	...summary,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
185	trendScore: Math.abs(summary.variationPercentage),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
186	badge:	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
187	summary.variationPercentage === 0	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
188	- "Monitorado"	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
189	: `\${isPriceDown - "-" : "+"}\${Math.abs(	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
190	summary.variationPercentage	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
191	).toFixed(1)}%`,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
192	badgeType:	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
193	summary.variationPercentage === 0	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
194	- "stable"	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
195	: isPriceDown	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Linha	Codigo	Explicacao
196	- "down"	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
197	: "up",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
198	};	Fecha bloco, objeto ou chamada anterior.
199	})	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
200	.sort((a, b) => {	Ordena itens antes de responder.
201	if (b.trendScore !== a.trendScore) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
202	return b.trendScore - a.trendScore;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
203	}	Fecha bloco, objeto ou chamada anterior.
204		Linha em branco para separar blocos e melhorar legibilidade.
205	return b.updatedAt.getTime() - a.updatedAt.getTime();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
206	})	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
207	.slice(0, query.limit [carrinho] 10);	Limita quantidade de resultados.
208	}	Fecha bloco, objeto ou chamada anterior.
209		Linha em branco para separar blocos e melhorar legibilidade.
210	async findOffers(id: number) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
211	const product = await prisma.product.findUnique({ where: { id } });	Acessa o banco pelo Prisma Client.
212		Linha em branco para separar blocos e melhorar legibilidade.
213	if (!product) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
214	throw new AppError("Product not found", 404);	Gera erro controlado com status HTTP.
215	}	Fecha bloco, objeto ou chamada anterior.
216		Linha em branco para separar blocos e melhorar legibilidade.
217	const offers = await prisma.product.findMany({	Acessa o banco pelo Prisma Client.
218	where: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
219	name: product.name,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
220	category: product.category,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
221	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
222	orderBy: [	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
223	{ currentPrice: "asc" },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
224	{ updatedAt: "desc" },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
225	],	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
226	});	Fecha bloco, objeto ou chamada anterior.
227		Linha em branco para separar blocos e melhorar legibilidade.
228	const cheapest = this.getCheapestProduct(offers);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Linha	Codigo	Explicacao
229		Linha em branco para separar blocos e melhorar legibilidade.
230	return {	Inicia objeto de resposta/retorno.
231	product: this.toProductSummary(offers),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
232	cheapestOfferId: cheapest.id,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
233	offers: offers.map((offer) => ({	Transforma itens de uma lista.
234	...this.toOffer(offer),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
235	isCheapest: offer.id === cheapest.id,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
236	))),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
237	};	Fecha bloco, objeto ou chamada anterior.
238	}	Fecha bloco, objeto ou chamada anterior.
239		Linha em branco para separar blocos e melhorar legibilidade.
240	async findPriceHistory(id: number, query: PriceHistoryQueryDto) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
241	const product = await prisma.product.findUnique({ where: { id } });	Acessa o banco pelo Prisma Client.
242		Linha em branco para separar blocos e melhorar legibilidade.
243	if (!product) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
244	throw new AppError("Product not found", 404);	Gera erro controlado com status HTTP.
245	}	Fecha bloco, objeto ou chamada anterior.
246		Linha em branco para separar blocos e melhorar legibilidade.
247	const since = new Date();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
248	since.setDate(since.getDate() - (query.days [carrinho] 90));	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
249		Linha em branco para separar blocos e melhorar legibilidade.
250	const relatedOffers = await prisma.product.findMany({	Acessa o banco pelo Prisma Client.
251	where: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
252	name: product.name,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
253	category: product.category,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
254	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
255	select: { id: true },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
256	});	Fecha bloco, objeto ou chamada anterior.
257		Linha em branco para separar blocos e melhorar legibilidade.
258	const relatedProductIds = relatedOffers.map((offer) => offer.id);	Transforma itens de uma lista.
259	const history = await prisma.priceHistory.findMany({	Acessa o banco pelo Prisma Client.
260	where: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
261	productId: { in: relatedProductIds },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Linha	Codigo	Explicacao
262	createdAt: { gte: since },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
263	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
264	include: { product: true },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
265	orderBy: { createdAt: "asc" },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
266	});	Fecha bloco, objeto ou chamada anterior.
267		Linha em branco para separar blocos e melhorar legibilidade.
268	return history.map((entry) => ({	Transforma itens de uma lista.
269	id: entry.id,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
270	productId: entry.productId,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
271	storeName: entry.product.store,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
272	price: entry.price,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
273	createdAt: entry.createdAt,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
274	}}});	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
275	}	Fecha bloco, objeto ou chamada anterior.
276		Linha em branco para separar blocos e melhorar legibilidade.
277	async recordPriceSnapshot(id: number, data: RecordPriceSnapshotDto) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
278	const product = await prisma.product.findUnique({ where: { id } });	Acessa o banco pelo Prisma Client.
279		Linha em branco para separar blocos e melhorar legibilidade.
280	if (!product) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
281	throw new AppError("Product not found", 404);	Gera erro controlado com status HTTP.
282	}	Fecha bloco, objeto ou chamada anterior.
283		Linha em branco para separar blocos e melhorar legibilidade.
284	if (data.price === product.currentPrice && data.externalId === undefined) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
285	return {	Inicia objeto de resposta/retorno.
286	...product,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
287	variationPercentage: this.calcularVariation(	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
288	product.currentPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
289	product.previousPrice	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
290	),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
291	changed: false,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
292	};	Fecha bloco, objeto ou chamada anterior.
293	}	Fecha bloco, objeto ou chamada anterior.
294		Linha em branco para separar blocos e melhorar legibilidade.

Linha	Codigo	Explicacao
295	const updatedProduct = await prisma.\$transaction(async (tx) => {	Acessa o banco pelo Prisma Client.
296	if (data.price !== product.currentPrice) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
297	await tx.priceHistory.create({	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
298	data: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
299	productId: id,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
300	price: product.currentPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
301	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
302	});	Fecha bloco, objeto ou chamada anterior.
303	}	Fecha bloco, objeto ou chamada anterior.
304		Linha em branco para separar blocos e melhorar legibilidade.
305	return tx.product.update({	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
306	where: { id },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
307	data: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
308	currentPrice: data.price,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
309	previousPrice:	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
310	data.price !== product.currentPrice	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
311	- product.currentPrice	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
312	: product.previousPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
313	...(data.externalId !== undefined && { externalId: data.externalId }},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
314	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
315	});	Fecha bloco, objeto ou chamada anterior.
316	});	Fecha bloco, objeto ou chamada anterior.
317		Linha em branco para separar blocos e melhorar legibilidade.
318	return {	Inicia objeto de resposta/retorno.
319	...updatedProduct,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
320	variationPercentage: this.calcularVariation(	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
321	updatedProduct.currentPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
322	updatedProduct.previousPrice	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
323	),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
324	changed: data.price !== product.currentPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
325	};	Fecha bloco, objeto ou chamada anterior.
326	}	Fecha bloco, objeto ou chamada anterior.
327		Linha em branco para separar blocos e melhorar legibilidade.

Linha	Codigo	Explicacao
328		Linha em branco para separar blocos e melhorar legibilidade.
329	<code>async findById(id: number) {</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
330	<code>const product = await prisma.product.findUnique({</code>	Acessa o banco pelo Prisma Client.
331	<code>where: { id },</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
332	<code>include: {</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
333	<code>priceHistory: { orderBy: { createdAt: "desc" } }</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
334	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
335	<code>});</code>	Fecha bloco, objeto ou chamada anterior.
336		Linha em branco para separar blocos e melhorar legibilidade.
337	<code>if (!product) {</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
338	<code>throw new AppError("Product not found", 404);</code>	Gera erro controlado com status HTTP.
339	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
340		Linha em branco para separar blocos e melhorar legibilidade.
341	<code>return {</code>	Inicia objeto de resposta/retorno.
342	<code>...product,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
343	<code>variationPercentage: this.calcularVariation(product.currentPrice, product.previousPrice),</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
344	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
345	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
346		Linha em branco para separar blocos e melhorar legibilidade.
347	<code>async create(data: CreateProductDto) {</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
348	<code>return prisma.product.create({</code>	Acessa o banco pelo Prisma Client.
349	<code>data: {</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
350	<code>...data,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
351	<code>previousPrice: data.currentPrice,</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
352	<code>},</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
353	<code>});</code>	Fecha bloco, objeto ou chamada anterior.
354	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
355		Linha em branco para separar blocos e melhorar legibilidade.
356	<code>async update(id: number, data: UpdateProductDto) {</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
357	<code>const product = await prisma.product.findUnique({ where: { id } });</code>	Acessa o banco pelo Prisma Client.
358		Linha em branco para separar blocos e melhorar legibilidade.
359	<code>if (!product) {</code>	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
360	<code>throw new AppError("Produto não encontrado", 404);</code>	Gera erro controlado com status HTTP.

Linha	Codigo	Explicacao
361	}	Fecha bloco, objeto ou chamada anterior.
362		Linha em branco para separar blocos e melhorar legibilidade.
363	const updateData: Prisma.ProductUpdateInput = { ...data };	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
364		Linha em branco para separar blocos e melhorar legibilidade.
365	if (data.currentPrice !== undefined && data.currentPrice !== product.currentPrice) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
366	updateData.previousPrice = product.currentPrice;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
367		Linha em branco para separar blocos e melhorar legibilidade.
368	await prisma.priceHistory.create({	Acessa o banco pelo Prisma Client.
369	data: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
370	productId: id,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
371	price: product.currentPrice	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
372	}	Fecha bloco, objeto ou chamada anterior.
373	});	Fecha bloco, objeto ou chamada anterior.
374	}	Fecha bloco, objeto ou chamada anterior.
375		Linha em branco para separar blocos e melhorar legibilidade.
376	const updatedProduct = await prisma.product.update({	Acessa o banco pelo Prisma Client.
377	where: { id },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
378	data: updateData,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
379	});	Fecha bloco, objeto ou chamada anterior.
380		Linha em branco para separar blocos e melhorar legibilidade.
381	return {	Inicia objeto de resposta/retorno.
382	...updatedProduct,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
383	variationPercentage: this.calcularVariation(updatedProduct.currentPrice, updatedProduct.previousPrice),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
384	};	Fecha bloco, objeto ou chamada anterior.
385	}	Fecha bloco, objeto ou chamada anterior.
386		Linha em branco para separar blocos e melhorar legibilidade.
387	async delete(id: number) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
388	const product = await prisma.product.findUnique({ where: { id } });	Acessa o banco pelo Prisma Client.
389		Linha em branco para separar blocos e melhorar legibilidade.
390	if (!product) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
391	throw new AppError("Produto não encontrado", 404);	Gera erro controlado com status HTTP.
392	}	Fecha bloco, objeto ou chamada anterior.
393		Linha em branco para separar blocos e melhorar legibilidade.

Linha	Codigo	Explicacao
394	<code>return prisma.product.delete({ where: { id } });</code>	Registra rota HTTP DELETE.
395	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
396		Linha em branco para separar blocos e melhorar legibilidade.
397	<code>}</code>	Fecha bloco, objeto ou chamada anterior.
398		Linha em branco para separar blocos e melhorar legibilidade.
399	<code>export const productService = new ProductService();</code>	Compos a estrutura, regra, tipo ou resposta desse arquivo.

src/services/external-product.service.ts

Regra que consulta provider externo e faz upsert no banco.

Linha	Codigo	Explicacao
1	import { Prisma, Product } from "@prisma/client";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	import { prisma } from "../prisma/client";	Importa dependencia, tipo ou modulo usado neste arquivo.
4	import { env } from "../config/env";	Importa dependencia, tipo ou modulo usado neste arquivo.
5	import { AppError } from "../errors/AppError";	Importa dependencia, tipo ou modulo usado neste arquivo.
6	import {	Importa dependencia, tipo ou modulo usado neste arquivo.
7	ExternalProductOffer,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8	ExternalProductProvider,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9	} from "../providers/external-product.provider";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	import { MercadoLivreProvider } from "../providers/mercado-livre.provider";	Importa dependencia, tipo ou modulo usado neste arquivo.
11	import {	Importa dependencia, tipo ou modulo usado neste arquivo.
12	ExternalProductSearchDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
13	ImportExternalProductsDto,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
14	} from "../dtos/external-products.dto";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
15		Linha em branco para separar blocos e melhorar legibilidade.
16	export class ExternalProductService {	Declara classe exportada usada por outras camadas.
17	private provider: ExternalProductProvider;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
18		Linha em branco para separar blocos e melhorar legibilidade.
19	constructor(provider = ExternalProductService.createProvider()) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
20	this.provider = provider;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
21	}	Fecha bloco, objeto ou chamada anterior.
22		Linha em branco para separar blocos e melhorar legibilidade.
23	private static createProvider(): ExternalProductProvider {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
24	if (env.EXTERNAL_PRODUCTS_PROVIDER === "mercadolivre") {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
25	return new MercadoLivreProvider();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
26	}	Fecha bloco, objeto ou chamada anterior.
27		Linha em branco para separar blocos e melhorar legibilidade.
28	throw new AppError("External products provider is not supported", 500);	Gera erro controlado com status HTTP.
29	}	Fecha bloco, objeto ou chamada anterior.
30		Linha em branco para separar blocos e melhorar legibilidade.

Linha	Codigo	Explicacao
31	private inferEmoji(product: ExternalProductOffer): string {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
32	const text = `\${product.name} \${product.category}`.toLowerCase();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
33		Linha em branco para separar blocos e melhorar legibilidade.
34	if (text.includes("phone") text.includes("iphone") text.includes("galaxy")) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
35	return " ";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
36	}	Fecha bloco, objeto ou chamada anterior.
37		Linha em branco para separar blocos e melhorar legibilidade.
38	if (text.includes("notebook") text.includes("macbook") text.includes("dell")) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
39	return " ";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
40	}	Fecha bloco, objeto ou chamada anterior.
41		Linha em branco para separar blocos e melhorar legibilidade.
42	if (text.includes("fone") text.includes("headphone") text.includes("audio")) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
43	return " ";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
44	}	Fecha bloco, objeto ou chamada anterior.
45		Linha em branco para separar blocos e melhorar legibilidade.
46	if (text.includes("playstation") text.includes("xbox") text.includes("nintendo")) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
47	return " ";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
48	}	Fecha bloco, objeto ou chamada anterior.
49		Linha em branco para separar blocos e melhorar legibilidade.
50	return " ";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
51	}	Fecha bloco, objeto ou chamada anterior.
52		Linha em branco para separar blocos e melhorar legibilidade.
53	private buildProductData(	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
54	product: ExternalProductOffer,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
55	category-: string	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
56	): Prisma.ProductCreateInput {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
57	return {	Inicia objeto de resposta/retorno.
58	name: product.name,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
59	store: product.store,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
60	category: category [carrinho] product.category,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
61	currentPrice: product.currentPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
62	previousPrice: product.currentPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
63	emoji: this.inferEmoji(product),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Linha	Codigo	Explicacao
64	externalId: product.externalId,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
65	};	Fecha bloco, objeto ou chamada anterior.
66	}	Fecha bloco, objeto ou chamada anterior.
67		Linha em branco para separar blocos e melhorar legibilidade.
68	private async upsertExternalProduct(	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
69	product: ExternalProductOffer,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
70	category-: string	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
71	): Promise<Product & { importedStatus: "created" "updated" "unchanged" }> {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
72	const existing = await prisma.product.findUnique({	Acessa o banco pelo Prisma Client.
73	where: { externalId: product.externalId },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
74	});	Fecha bloco, objeto ou chamada anterior.
75		Linha em branco para separar blocos e melhorar legibilidade.
76	if (!existing) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
77	const created = await prisma.product.create({	Acessa o banco pelo Prisma Client.
78	data: this.buildProductData(product, category),	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
79	});	Fecha bloco, objeto ou chamada anterior.
80		Linha em branco para separar blocos e melhorar legibilidade.
81	return { ...created, importedStatus: "created" };	Inicia objeto de resposta/retorno.
82	}	Fecha bloco, objeto ou chamada anterior.
83		Linha em branco para separar blocos e melhorar legibilidade.
84	const priceChanged = existing.currentPrice !== product.currentPrice;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
85	const productData = this.buildProductData(product, category);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
86		Linha em branco para separar blocos e melhorar legibilidade.
87	const updated = await prisma.\$transaction(async (tx) => {	Acessa o banco pelo Prisma Client.
88	if (priceChanged) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
89	await tx.priceHistory.create({	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
90	data: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
91	productId: existing.id,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
92	price: existing.currentPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
93	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
94	});	Fecha bloco, objeto ou chamada anterior.
95	}	Fecha bloco, objeto ou chamada anterior.
96		Linha em branco para separar blocos e melhorar legibilidade.

Linha	Codigo	Explicacao
97	return tx.product.update({	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
98	where: { id: existing.id },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
99	data: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
100	name: productData.name,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
101	store: productData.store,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
102	category: productData.category,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
103	currentPrice: product.currentPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
104	previousPrice: priceChanged	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
105	- existing.currentPrice	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
106	: existing.previousPrice,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
107	emoji: productData.emoji,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
108	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
109	});	Fecha bloco, objeto ou chamada anterior.
110	});	Fecha bloco, objeto ou chamada anterior.
111		Linha em branco para separar blocos e melhorar legibilidade.
112	return {	Inicia objeto de resposta/retorno.
113	...updated,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
114	importedStatus: priceChanged - "updated" : "unchanged",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
115	};	Fecha bloco, objeto ou chamada anterior.
116	}	Fecha bloco, objeto ou chamada anterior.
117		Linha em branco para separar blocos e melhorar legibilidade.
118	async search(query: ExternalProductSearchDto) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
119	return this.provider.search({	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
120	query: query.q,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
121	limit: query.limit [carrinho] 10,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
122	});	Fecha bloco, objeto ou chamada anterior.
123	}	Fecha bloco, objeto ou chamada anterior.
124		Linha em branco para separar blocos e melhorar legibilidade.
125	async importProducts(data: ImportExternalProductsDto) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
126	const externalProducts = await this.provider.search({	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
127	query: data.q,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
128	limit: data.limit [carrinho] 10,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
129	});	Fecha bloco, objeto ou chamada anterior.

Linha	Codigo	Explicacao
130		Linha em branco para separar blocos e melhorar legibilidade.
131	const importedProducts = [];	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
132		Linha em branco para separar blocos e melhorar legibilidade.
133	for (const product of externalProducts) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
134	importedProducts.push(	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
135	await this.upsertExternalProduct(product, data.category)	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
136	);	Fecha bloco, objeto ou chamada anterior.
137	}	Fecha bloco, objeto ou chamada anterior.
138		Linha em branco para separar blocos e melhorar legibilidade.
139	return {	Inicia objeto de resposta/retorno.
140	provider: env.EXTERNAL_PRODUCTS_PROVIDER,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
141	query: data.q,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
142	imported: importedProducts.length,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
143	created: importedProducts.filter((product) => product.importedStatus === "created")	Filtra itens de uma lista.
144	.length,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
145	updated: importedProducts.filter((product) => product.importedStatus === "updated")	Filtra itens de uma lista.
146	.length,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
147	unchanged: importedProducts.filter(	Filtra itens de uma lista.
148	(product) => product.importedStatus === "unchanged"	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
149	).length,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
150	products: importedProducts,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
151	};	Fecha bloco, objeto ou chamada anterior.
152	}	Fecha bloco, objeto ou chamada anterior.
153	}	Fecha bloco, objeto ou chamada anterior.

src/services/alert.service.ts

Reservado para alertas; atualmente vazio.

Este arquivo esta vazio no estado atual do projeto. Ele existe como ponto reservado para implementacao futura.

src/providers/external-product.provider.ts

Contrato generico de provider externo.

Linha	Codigo	Explicacao
1	export interface ExternalProductSearchParams {	Define contrato de dados TypeScript.
2	query: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
3	limit: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
4	}	Fecha bloco, objeto ou chamada anterior.
5		Linha em branco para separar blocos e melhorar legibilidade.
6	export interface ExternalProductOffer {	Define contrato de dados TypeScript.
7	externalId: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8	name: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9	store: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	category: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
11	currentPrice: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
12	currency: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
13	productUrl?: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
14	imageUrl?: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
15	}	Fecha bloco, objeto ou chamada anterior.
16		Linha em branco para separar blocos e melhorar legibilidade.
17	export interface ExternalProductProvider {	Define contrato de dados TypeScript.
18	search(params: ExternalProductSearchParams): Promise<ExternalProductOffer[]>;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
19	}	Fecha bloco, objeto ou chamada anterior.

src/providers/mercado-livre.provider.ts

Provider concreto do Mercado Livre.

Linha	Codigo	Explicacao
1	import { AppError } from "../errors/AppError";	Importa dependencia, tipo ou modulo usado neste arquivo.
2	import { env } from "../config/env";	Importa dependencia, tipo ou modulo usado neste arquivo.
3	import {	Importa dependencia, tipo ou modulo usado neste arquivo.
4	ExternalProductOffer,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
5	ExternalProductProvider,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
6	ExternalProductSearchParams,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
7	} from "../external-product.provider";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8		Linha em branco para separar blocos e melhorar legibilidade.
9	interface MercadoLivreSearchResponse {	Define contrato de dados TypeScript.
10	results-: MercadoLivreItem[];	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
11	}	Fecha bloco, objeto ou chamada anterior.
12		Linha em branco para separar blocos e melhorar legibilidade.
13	interface MercadoLivreItem {	Define contrato de dados TypeScript.
14	id: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
15	title: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
16	price: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
17	currency_id: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
18	category_id: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
19	permalink-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
20	thumbnail-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
21	seller-: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
22	nickname-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
23	};	Fecha bloco, objeto ou chamada anterior.
24	}	Fecha bloco, objeto ou chamada anterior.
25		Linha em branco para separar blocos e melhorar legibilidade.
26	export class MercadoLivreProvider implements ExternalProductProvider {	Declara classe exportada usada por outras camadas.
27	async search(params: ExternalProductSearchParams): Promise<ExternalProductOffer[]> {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
28	const url = new URL(	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
29	`/sites/\${env.MERCADO_LIVRE_SITE_ID}/search`,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
30	env.MERCADO_LIVRE_API_BASE_URL	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Linha	Codigo	Explicacao
31	);	Fecha bloco, objeto ou chamada anterior.
32		Linha em branco para separar blocos e melhorar legibilidade.
33	url.searchParams.set("q", params.query);	Adiciona parametros na URL externa.
34	url.searchParams.set("limit", String(params.limit));	Adiciona parametros na URL externa.
35		Linha em branco para separar blocos e melhorar legibilidade.
36	const response = await fetch(url, {	Faz chamada HTTP para API externa.
37	headers: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
38	Accept: "application/json",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
39	"User-Agent": "mobile-price-comparator-api/1.0",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
40	},	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
41	});	Fecha bloco, objeto ou chamada anterior.
42		Linha em branco para separar blocos e melhorar legibilidade.
43	if (!response.ok) {	Verifica sucesso da resposta externa.
44	throw new AppError(	Gera erro controlado com status HTTP.
45	`External product provider failed with status \${response.status}`,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
46	502	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
47	);	Fecha bloco, objeto ou chamada anterior.
48	}	Fecha bloco, objeto ou chamada anterior.
49		Linha em branco para separar blocos e melhorar legibilidade.
50	const payload = (await response.json()) as MercadoLivreSearchResponse;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
51		Linha em branco para separar blocos e melhorar legibilidade.
52	return (payload.results [carrinho] []).map((item) => ({	Transforma itens de uma lista.
53	externalId: `mercadolivre:\${item.id}`,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
54	name: item.title,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
55	store: item.seller-.nickname [carrinho] "Mercado Livre",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
56	category: item.category_id,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
57	currentPrice: item.price,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
58	currency: item.currency_id,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
59	productUrl: item.permalink,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
60	imageUrl: item.thumbnail,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
61	}}));	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
62	}	Fecha bloco, objeto ou chamada anterior.
63	}	Fecha bloco, objeto ou chamada anterior.

src/dtos/create-user.dto.ts

Valida dados de cadastro.

Linha	Codigo	Explicacao
1	import { IsEmail, IsNotEmpty, MinLength } from "class-validator";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	export class CreateUserDto {	Declara classe exportada usada por outras camadas.
4	@IsNotEmpty()	Decorator de validacao/transformacao do DTO.
5	name!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
6		Linha em branco para separar blocos e melhorar legibilidade.
7	@IsEmail()	Decorator de validacao/transformacao do DTO.
8	email!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9		Linha em branco para separar blocos e melhorar legibilidade.
10	@MinLength(6)	Decorator de validacao/transformacao do DTO.
11	password!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
12	}	Fecha bloco, objeto ou chamada anterior.

src/dtos/login.dto.ts

Tipo de login usado pelo AuthService.

Linha	Codigo	Explicacao
1	export interface LoginDto {	Define contrato de dados TypeScript.
2	login: string,	Compos a estrutura, regra, tipo ou resposta desse arquivo.
3	password: string	Compos a estrutura, regra, tipo ou resposta desse arquivo.
4	}	Fecha bloco, objeto ou chamada anterior.

src/dtos/product-list.dto.ts

Filtros do list tradicional.

Linha	Codigo	Explicacao
1	import { IsOptional, IsString, IsNumber } from "class-validator";	Importa dependencia, tipo ou modulo usado neste arquivo.
2	import { Type } from "class-transformer";	Importa dependencia, tipo ou modulo usado neste arquivo.
3		Linha em branco para separar blocos e melhorar legibilidade.
4	export class ProductListDto {	Declara classe exportada usada por outras camadas.
5	@IsOptional()	Decorator de validacao/transformacao do DTO.
6	@IsString()	Decorator de validacao/transformacao do DTO.
7	name-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8		Linha em branco para separar blocos e melhorar legibilidade.
9	@IsOptional()	Decorator de validacao/transformacao do DTO.
10	@IsString()	Decorator de validacao/transformacao do DTO.
11	store-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
12		Linha em branco para separar blocos e melhorar legibilidade.
13	@IsOptional()	Decorator de validacao/transformacao do DTO.
14	@IsString()	Decorator de validacao/transformacao do DTO.
15	category-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
16		Linha em branco para separar blocos e melhorar legibilidade.
17	@IsOptional()	Decorator de validacao/transformacao do DTO.
18	@Type(() => Number)	Decorator de validacao/transformacao do DTO.
19	@IsNumber({}, { message: "minPrice must be a valid number"})	Decorator de validacao/transformacao do DTO.
20	minPrice-: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
21		Linha em branco para separar blocos e melhorar legibilidade.
22	@IsOptional()	Decorator de validacao/transformacao do DTO.
23	@Type(() => Number)	Decorator de validacao/transformacao do DTO.
24	@IsNumber({}, { message: "maxPrice must be a valid number"})	Decorator de validacao/transformacao do DTO.
25	maxPrice-: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
26	}	Fecha bloco, objeto ou chamada anterior.

src/dtos/product.dto.ts

DTOs de produto, busca, trending, historico e snapshot.

Linha	Codigo	Explicacao
1	import {	Importa dependencia, tipo ou modulo usado neste arquivo.
2	IsInt,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
3	IsNumber,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
4	IsOptional,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
5	IsPositive,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
6	IsString,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
7	Max,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8	Min,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9	} from "class-validator";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	import { Type } from "class-transformer";	Importa dependencia, tipo ou modulo usado neste arquivo.
11		Linha em branco para separar blocos e melhorar legibilidade.
12	export class IdParamDto {	Declara classe exportada usada por outras camadas.
13	@Type(() => Number)	Decorator de validacao/transformacao do DTO.
14	@IsInt({ message: "ID must be a valid integer number" })	Decorator de validacao/transformacao do DTO.
15	@IsPositive({ message: "ID must be a number greater than zero" })	Decorator de validacao/transformacao do DTO.
16	id!: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
17	}	Fecha bloco, objeto ou chamada anterior.
18		Linha em branco para separar blocos e melhorar legibilidade.
19	export class CreateProductDto {	Declara classe exportada usada por outras camadas.
20	@IsString({ message: "Name is required" })	Decorator de validacao/transformacao do DTO.
21	name!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
22		Linha em branco para separar blocos e melhorar legibilidade.
23	@IsString({ message: "Emoji must be a String" })	Decorator de validacao/transformacao do DTO.
24	emoji!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
25		Linha em branco para separar blocos e melhorar legibilidade.
26	@IsString({ message: "Store is required" })	Decorator de validacao/transformacao do DTO.
27	store!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
28		Linha em branco para separar blocos e melhorar legibilidade.
29	@IsString({ message: "Category is required" })	Decorator de validacao/transformacao do DTO.
30	category!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Linha	Codigo	Explicacao
31		Linha em branco para separar blocos e melhorar legibilidade.
32	@IsNumber({}, { message: "Price must be a number" })	Decorator de validacao/transformacao do DTO.
33	@IsPositive({ message: "Price must be greater than zero" })	Decorator de validacao/transformacao do DTO.
34	currentPrice!: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
35		Linha em branco para separar blocos e melhorar legibilidade.
36	@IsOptional()	Decorator de validacao/transformacao do DTO.
37	@IsString({ message: "External ID must be a string" })	Decorator de validacao/transformacao do DTO.
38	externalId!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
39	}	Fecha bloco, objeto ou chamada anterior.
40		Linha em branco para separar blocos e melhorar legibilidade.
41	export class UpdateProductDto {	Declara classe exportada usada por outras camadas.
42	@IsString()	Decorator de validacao/transformacao do DTO.
43	name!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
44		Linha em branco para separar blocos e melhorar legibilidade.
45	@IsString()	Decorator de validacao/transformacao do DTO.
46	store!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
47		Linha em branco para separar blocos e melhorar legibilidade.
48	@IsString()	Decorator de validacao/transformacao do DTO.
49	category!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
50		Linha em branco para separar blocos e melhorar legibilidade.
51	@IsNumber()	Decorator de validacao/transformacao do DTO.
52	@IsPositive()	Decorator de validacao/transformacao do DTO.
53	currentPrice!: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
54		Linha em branco para separar blocos e melhorar legibilidade.
55	@IsString()	Decorator de validacao/transformacao do DTO.
56	emoji!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
57		Linha em branco para separar blocos e melhorar legibilidade.
58	@IsOptional()	Decorator de validacao/transformacao do DTO.
59	@IsString()	Decorator de validacao/transformacao do DTO.
60	externalId!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
61	}	Fecha bloco, objeto ou chamada anterior.
62		Linha em branco para separar blocos e melhorar legibilidade.
63	export class ProductSearchDto {	Declara classe exportada usada por outras camadas.

Linha	Codigo	Explicacao
64	@IsOptional()	Decorator de validacao/transformacao do DTO.
65	@IsString()	Decorator de validacao/transformacao do DTO.
66	q-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
67		Linha em branco para separar blocos e melhorar legibilidade.
68	@IsOptional()	Decorator de validacao/transformacao do DTO.
69	@IsString()	Decorator de validacao/transformacao do DTO.
70	store-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
71		Linha em branco para separar blocos e melhorar legibilidade.
72	@IsOptional()	Decorator de validacao/transformacao do DTO.
73	@IsString()	Decorator de validacao/transformacao do DTO.
74	category-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
75		Linha em branco para separar blocos e melhorar legibilidade.
76	@IsOptional()	Decorator de validacao/transformacao do DTO.
77	@Type(() => Number)	Decorator de validacao/transformacao do DTO.
78	@IsNumber({}, { message: "minPrice must be a valid number" })	Decorator de validacao/transformacao do DTO.
79	minPrice-: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
80		Linha em branco para separar blocos e melhorar legibilidade.
81	@IsOptional()	Decorator de validacao/transformacao do DTO.
82	@Type(() => Number)	Decorator de validacao/transformacao do DTO.
83	@IsNumber({}, { message: "maxPrice must be a valid number" })	Decorator de validacao/transformacao do DTO.
84	maxPrice-: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
85		Linha em branco para separar blocos e melhorar legibilidade.
86	@IsOptional()	Decorator de validacao/transformacao do DTO.
87	@Type(() => Number)	Decorator de validacao/transformacao do DTO.
88	@IsInt({ message: "limit must be an integer" })	Decorator de validacao/transformacao do DTO.
89	@Min(1, { message: "limit must be greater than zero" })	Decorator de validacao/transformacao do DTO.
90	@Max(50, { message: "limit must be at most 50" })	Decorator de validacao/transformacao do DTO.
91	limit-: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
92	}	Fecha bloco, objeto ou chamada anterior.
93		Linha em branco para separar blocos e melhorar legibilidade.
94	export class ProductTrendingDto {	Declara classe exportada usada por outras camadas.
95	@IsOptional()	Decorator de validacao/transformacao do DTO.
96	@Type(() => Number)	Decorator de validacao/transformacao do DTO.

Linha	Codigo	Explicacao
97	@IsInt({ message: "limit must be an integer" })	Decorator de validacao/transformacao do DTO.
98	@Min(1, { message: "limit must be greater than zero" })	Decorator de validacao/transformacao do DTO.
99	@Max(50, { message: "limit must be at most 50" })	Decorator de validacao/transformacao do DTO.
100	limit-: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
101	}	Fecha bloco, objeto ou chamada anterior.
102		Linha em branco para separar blocos e melhorar legibilidade.
103	export class PriceHistoryQueryDto {	Declara classe exportada usada por outras camadas.
104	@IsOptional()	Decorator de validacao/transformacao do DTO.
105	@Type(() => Number)	Decorator de validacao/transformacao do DTO.
106	@IsInt({ message: "days must be an integer" })	Decorator de validacao/transformacao do DTO.
107	@Min(1, { message: "days must be greater than zero" })	Decorator de validacao/transformacao do DTO.
108	@Max(365, { message: "days must be at most 365" })	Decorator de validacao/transformacao do DTO.
109	days-: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
110	}	Fecha bloco, objeto ou chamada anterior.
111		Linha em branco para separar blocos e melhorar legibilidade.
112	export class RecordPriceSnapshotDto {	Declara classe exportada usada por outras camadas.
113	@Type(() => Number)	Decorator de validacao/transformacao do DTO.
114	@IsNumber({}, { message: "Price must be a number" })	Decorator de validacao/transformacao do DTO.
115	@IsPositive({ message: "Price must be greater than zero" })	Decorator de validacao/transformacao do DTO.
116	price!: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
117		Linha em branco para separar blocos e melhorar legibilidade.
118	@IsOptional()	Decorator de validacao/transformacao do DTO.
119	@IsString()	Decorator de validacao/transformacao do DTO.
120	externalId-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
121	}	Fecha bloco, objeto ou chamada anterior.
122		Linha em branco para separar blocos e melhorar legibilidade.

src/dtos/external-products.dto.ts

DTOs da busca/importacao externa.

Linha	Codigo	Explicacao
1	import { IsInt, IsNotEmpty, IsOptional, IsString, Max, Min } from "class-validator";	Importa dependencia, tipo ou modulo usado neste arquivo.
2	import { Type } from "class-transformer";	Importa dependencia, tipo ou modulo usado neste arquivo.
3		Linha em branco para separar blocos e melhorar legibilidade.
4	export class ExternalProductSearchDto {	Declara classe exportada usada por outras camadas.
5	@IsString({ message: "Search query is required" })	Decorator de validacao/transformacao do DTO.
6	@IsNotEmpty({ message: "Search query cannot be empty" })	Decorator de validacao/transformacao do DTO.
7	q!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8		Linha em branco para separar blocos e melhorar legibilidade.
9	@IsOptional()	Decorator de validacao/transformacao do DTO.
10	@Type(() => Number)	Decorator de validacao/transformacao do DTO.
11	@IsInt({ message: "limit must be an integer" })	Decorator de validacao/transformacao do DTO.
12	@Min(1, { message: "limit must be greater than zero" })	Decorator de validacao/transformacao do DTO.
13	@Max(20, { message: "limit must be at most 20" })	Decorator de validacao/transformacao do DTO.
14	limit-: number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
15	}	Fecha bloco, objeto ou chamada anterior.
16		Linha em branco para separar blocos e melhorar legibilidade.
17	export class ImportExternalProductsDto {	Declara classe exportada usada por outras camadas.
18	@IsString({ message: "Search query is required" })	Decorator de validacao/transformacao do DTO.
19	@IsNotEmpty({ message: "Search query cannot be empty" })	Decorator de validacao/transformacao do DTO.
20	q!: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
21		Linha em branco para separar blocos e melhorar legibilidade.
22	@IsOptional()	Decorator de validacao/transformacao do DTO.
23	@IsString()	Decorator de validacao/transformacao do DTO.
24	category-: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
25		Linha em branco para separar blocos e melhorar legibilidade.
26	@IsOptional()	Decorator de validacao/transformacao do DTO.
27	@Type(() => Number)	Decorator de validacao/transformacao do DTO.
28	@IsInt({ message: "limit must be an integer" })	Decorator de validacao/transformacao do DTO.
29	@Min(1, { message: "limit must be greater than zero" })	Decorator de validacao/transformacao do DTO.
30	@Max(20, { message: "limit must be at most 20" })	Decorator de validacao/transformacao do DTO.

Linha	Codigo	Explicacao
31	limit-: number;	Compos a estrutura, regra, tipo ou resposta desse arquivo.
32	}	Fecha bloco, objeto ou chamada anterior.

src/dtos/create-alert.dto.ts

Reservado para criacao de alerta; vazio.

Este arquivo esta vazio no estado atual do projeto. Ele existe como ponto reservado para implementacao futura.

src/dtos/update-alert.dto.ts

Reservado para atualizacao de alerta; vazio.

Este arquivo esta vazio no estado atual do projeto. Ele existe como ponto reservado para implementacao futura.

src/middleware/validation.middleware.ts

Valida body, query e params com DTOs.

Linha	Codigo	Explicacao
1	import { ClassConstructor, plainToInstance } from "class-transformer";	Importa dependencia, tipo ou modulo usado neste arquivo.
2	import { validate, ValidationError } from "class-validator";	Importa dependencia, tipo ou modulo usado neste arquivo.
3	import { Request, Response, NextFunction } from "express";	Importa dependencia, tipo ou modulo usado neste arquivo.
4	import { AppError } from "../errors/AppError";	Importa dependencia, tipo ou modulo usado neste arquivo.
5		Linha em branco para separar blocos e melhorar legibilidade.
6	type RequestSource = "body" "query" "params";	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
7		Linha em branco para separar blocos e melhorar legibilidade.
8	export function validateDto<T extends object> (	Aplica validacao antes do controller.
9	dtoClass: ClassConstructor<T>,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	source: RequestSource = "body"	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
11	) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
12	return async (req: Request, res: Response, next: NextFunction) => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
13	const instance = plainToInstance(dtoClass, req[source]);	Transforma objeto bruto em instancia de DTO.
14		Linha em branco para separar blocos e melhorar legibilidade.
15	const errors: ValidationError[] = await validate(instance, {	Executa validacoes do class-validator.
16	whitelist: true,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
17	forbidNonWhitelisted: false,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
18	});	Fecha bloco, objeto ou chamada anterior.
19		Linha em branco para separar blocos e melhorar legibilidade.
20	if (errors.length > 0) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
21	const errorMessages = errors	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
22	.map((error: ValidationError) => Object.values(error.constraints {}).join(", "))	Transforma itens de uma lista.
23	.join("; ");	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
24		Linha em branco para separar blocos e melhorar legibilidade.
25	return next(new AppError(errorMessages, 400));	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
26	}	Fecha bloco, objeto ou chamada anterior.
27		Linha em branco para separar blocos e melhorar legibilidade.
28	req[source] = instance;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
29	next();	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
30	}	Fecha bloco, objeto ou chamada anterior.

Linha	Codigo	Explicacao
31	}	Fecha bloco, objeto ou chamada anterior.

src/middleware/auth.middleware.ts

Reservado para autenticação JWT; vazio.

Este arquivo está vazio no estado atual do projeto. Ele existe como ponto reservado para implementação futura.

src/errors/AppError.ts

Erro controlado com status HTTP.

Linha	Codigo	Explicacao
1	<code>export class AppError extends Error {</code>	Declara classe exportada usada por outras camadas.
2	<code>public readonly statusCode: number;</code>	Compos a estrutura, regra, tipo ou resposta desse arquivo.
3		Linha em branco para separar blocos e melhorar legibilidade.
4	<code>constructor(message: string, statusCode = 400) {</code>	Compos a estrutura, regra, tipo ou resposta desse arquivo.
5	<code>super(message);</code>	Compos a estrutura, regra, tipo ou resposta desse arquivo.
6		Linha em branco para separar blocos e melhorar legibilidade.
7	<code>this.statusCode = statusCode;</code>	Compos a estrutura, regra, tipo ou resposta desse arquivo.
8		Linha em branco para separar blocos e melhorar legibilidade.
9	<code>Object.setPrototypeOf(this, AppError.prototype);</code>	Compos a estrutura, regra, tipo ou resposta desse arquivo.
10	<code>}</code>	Fechou bloco, objeto ou chamada anterior.
11	<code>}</code>	Fechou bloco, objeto ou chamada anterior.

src/errors/errorHandler.ts

Middleware final que converte erros em JSON.

Linha	Codigo	Explicacao
1	import { Request, Response, NextFunction } from "express";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	import { AppError } from "../AppError";	Importa dependencia, tipo ou modulo usado neste arquivo.
4		Linha em branco para separar blocos e melhorar legibilidade.
5	export const errorHandler = (	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
6	error: Error,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
7	req: Request,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8	res: Response,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9	next: NextFunction	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	): Response => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
11	if (error instanceof AppError) {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
12	return res.status(error.statusCode).json({	Define status HTTP e envia resposta.
13	success: false,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
14	message: error.message,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
15	});	Fecha bloco, objeto ou chamada anterior.
16	}	Fecha bloco, objeto ou chamada anterior.
17		Linha em branco para separar blocos e melhorar legibilidade.
18	console.error(error);	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
19		Linha em branco para separar blocos e melhorar legibilidade.
20	return res.status(500).json({	Define status HTTP e envia resposta.
21	success: false,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
22	message: "Internal server error",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
23	});	Fecha bloco, objeto ou chamada anterior.
24	};	Fecha bloco, objeto ou chamada anterior.

src/utls/bcrypt.ts

Hash e comparacao de senhas.

Linha	Codigo	Explicacao
1	import bcrypt from "bcryptjs";	Importa dependencia, tipo ou modulo usado neste arquivo.
2		Linha em branco para separar blocos e melhorar legibilidade.
3	const SALT_ROUNDS = 10;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
4		Linha em branco para separar blocos e melhorar legibilidade.
5	export const hashPassword = async (password: string): Promise<string> => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
6	return bcrypt.hash(password, SALT_ROUNDS);	Gera hash seguro da senha.
7	};	Fecha bloco, objeto ou chamada anterior.
8		Linha em branco para separar blocos e melhorar legibilidade.
9	export const comparePassword = async (	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	password: string,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
11	hashedPassword: string	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
12	): Promise<boolean> => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
13	return bcrypt.compare(password, hashedPassword);	Compara senha com hash salvo.
14	};	Fecha bloco, objeto ou chamada anterior.

src/utils/jwt.ts

Geracao e validacao de JWT.

Linha	Codigo	Explicacao
1	import jwt from "jsonwebtoken";	Importa dependencia, tipo ou modulo usado neste arquivo.
2	import { env } from "../config/env";	Importa dependencia, tipo ou modulo usado neste arquivo.
3		Linha em branco para separar blocos e melhorar legibilidade.
4	export const generateToken = (userId: number): string => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
5	return jwt.sign(	Gera token JWT.
6	{ userId },	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
7	env.JWT_SECRET,	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
8	{	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
9	expiresIn: "7d",	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
10	}	Fecha bloco, objeto ou chamada anterior.
11	);	Fecha bloco, objeto ou chamada anterior.
12	};	Fecha bloco, objeto ou chamada anterior.
13		Linha em branco para separar blocos e melhorar legibilidade.
14	export const verifyToken = (token: string) => {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
15	return jwt.verify(token, env.JWT_SECRET);	Valida token JWT.
16	};	Fecha bloco, objeto ou chamada anterior.

src/types/express.d.ts

Extensao de tipos para req.user.

Linha	Codigo	Explicacao
1	declare global {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
2	namespace Express {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
3	interface Request {	Define contrato de dados TypeScript.
4	user-: {	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
5	id: string number;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
6	email: string;	Compoe a estrutura, regra, tipo ou resposta desse arquivo.
7	};	Fecha bloco, objeto ou chamada anterior.
8	}	Fecha bloco, objeto ou chamada anterior.
9	}	Fecha bloco, objeto ou chamada anterior.
10	}	Fecha bloco, objeto ou chamada anterior.
11		Linha em branco para separar blocos e melhorar legibilidade.
12	export {};	Compoe a estrutura, regra, tipo ou resposta desse arquivo.

Como apresentar este projeto

Explique que a API separa responsabilidades: rotas recebem, DTOs validam, controllers encaminham, services decidem, Prisma persiste e providers conversam com sistemas externos. O ponto principal do comparador e que nossa API importa, normaliza, guarda historico e entrega dados estaveis para o app.